IF3170 INTELIGENSI ARTIFISIAL

LAPORAN TUGAS BESAR 1



Disusun oleh:

Kelompok 28 - MindfulnessAI

Renuno Yuqa Frinardi	(13524080)
Valentino Daniel Kusumo	(13524104)
Leonardus Brain Fatolosja	(13524146)

Dikumpulkan pada:

20 Oktober 2026

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI **2**
BAB I
DESKRIPSI PERSOALAN **4**
1. Latar Belakang 4
2. Representasi State 4
3. Tujuan 5
BAB II
PERANCANGAN DAN IMPLEMENTASI **6**
1. Hill Climbing Algorithm 6
1.1. Pengertian Steepest Ascent Hill Climbing 6
1.1.1. Pengertian 6
1.1.2. Pseudocode 6
1.1.3. Implementasi 7
1.2. Sideways Move Hill Climbing 9
1.2.1. Pengertian 9
1.2.2. Pseudocode 9
1.2.3. Implementasi 10
1.3. Random Restart Hill Climbing 12
1.3.1. Pengertian 12
1.3.2. Pseudocode 13
1.3.3. Implementasi 13
1.4. Stochastic Hill Climbing 16
1.4.1. Pengertian 16
1.4.2. Pseudocode 16
1.4.3. Implementasi 16
2. Simulated Annealing Algorithm 18
2.1. Pengertian 18
2.2. Pseudocode 18
2.3. Implementasi 19
3. Genetic Algorithm 21
3.1. Pengertian Genetic Algorithm 21
3.2. Alur Program dan Pseudocode 21
3.3. Implementasi 24
4. Objective Function **28**
4.1. Objective Function 1 (Wajib) 28
4.2. Objective Function 2 (Bonus) 28
5. Multitruk (Bonus) **28**
5.1. Penjelasan Alur 28
5.2. Implementasi 29
BAB III
ANALISIS DAN PEMBAHASAN **30**
1. Hill-Climbing Algorithm 30
2. Simulated Annealing 42

3. Genetic Algorithm (Banyak Iterasi) 44
4. Genetic Algorithm (Jumlah Populasi) 47
5. Objective Value 1 50
6. Objective Value 2 52
7. Multitruk 53
8. Ringkasan Perbandingan 55
9. Pembahasan Perbandingan 56
a. Tabel 1 - Perbandingan enam Algoritma 56
b. Tabel 2 - Perbandingan dua Objective Value 57
c. Tabel 3 - Perbandingan dua Algoritma pada multitruk 57
BAB IV
KESIMPULAN DAN SARAN 59
1. Kesimpulan 59
2. Saran 59
PEMBAGIAN TUGAS 60
REFERENSI 61
LAMPIRAN FORM PENGGUNAAN GENERATIVE AI 62
LAMPIRAN 63

BAB I

DESKRIPSI PERSOALAN

1. Latar Belakang

Persoalan yang diselesaikan adalah pemuatan paket ke dalam sebuah truk. Truk memiliki ruang muat berbentuk balok dan batas kapasitas berat. Setiap paket memiliki dimensi, berat, nilai, penanda mudah pecah (fragile), serta waktu tiba (ETA). Tidak semua paket dapat dimuat sehingga harus dipilih *list* paket beserta posisi dan orientasinya agar total nilai muatan bernilai maksimum tanpa melanggar batasan.

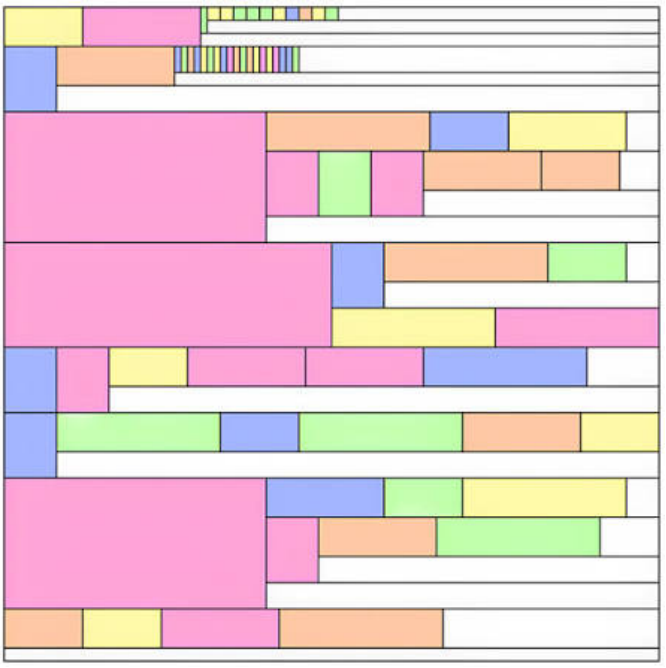
Entitas	Atribut	Contoh	Keterangan
Package	id	"RusdiPackage"	*Identifier*
	Dimensions	{w:3, l:4, h:1}	Ukuran sebuah paket, yaitu *width*, *length*, dan *height*
	Orientation	{w:x, l:y, h:z}	Sumbu yang sejajar dengan setiap sisi paket. Terdapat enam kemungkinan orientasi, misalnya {w:y, l:x, h:z} setelah rotasi 90° terhadap sumbu *z*.
	Value	60	Nilai harga sebuah paket (satuan unit)
	Weight	5	Berat paket (satuan unit berat)
	IsFragile	False	Benda pecah-belah
	ETA	3	Estimated Time of Arrival, perkiraan waktu kedatangan paket (dalam sisa hari)
Truck	Dimensions	{w:10, l:20, h:5}	Ukuran sebuah peti kemas pada truk, yaitu *width*, *length*, dan *height*
	MaxCapacity	500	Kapasitas angkut maksimum pada suatu peti kemas (satuan unit berat)

2. Representasi State

- **State**: konfigurasi muatan truk, yaitu paket mana yang dimuat, posisinya, dan orientasinya.
- **State awal**; seluruh paket berada di luar truk, kemudian dimasukkan satu per satu melalui proses pencarian lokal.
- **Neighbour**: mengubah *state* dengan melakukan berbagai operasi, seperti memasukkan paket, mengeluarkan paket, menggeser posisi, memutar orientasi, atau menukar 2 paket
- **Fungsi Objektif**: total nilai paket yang dimuat dalam truk. *State* yang melanggar batasan bernilai *-inf* agar tidak pernah diterima
- **Batasan**: paket tidak boleh melewati dinding truk, tidak boleh saling tumpang tindih, total berat tidak melebihi kapasitas

3. Tujuan

Persoalan ini memiliki tujuan, yakni menentukan posisi urutan paket sedemikian rupa agar memenuhi *objective function* sebaik mungkin. Menemukan konfigurasi muatan dengan nilai fungsi objektif setinggi mungkin menggunakan 3 algoritma pencarian lokal, lalu membandingkan solusi, waktu eksekusi, dan perilakunya



Gambar 1. Contoh penempatan paket agar terstruktur

BAB II

PERANCANGAN DAN IMPLEMENTASI

1. Hill Climbing Algorithm

1.1. Pengertian Steepest Ascent Hill Climbing

1.1.1. Pengertian

Steepest Ascent Hill Climbing merupakan varian algoritma local search yang bekerja secara berulang (iteratif) untuk meningkatkan kualitas solusi dengan selalu bergerak menuju state tetangga (neighbor) terbaik. Pada setiap iterasi, algoritma ini membangkitkan sejumlah kandidat neighbor dari state saat ini, mengevaluasi seluruh kandidat tersebut, lalu memilih neighbor dengan nilai objective (value) paling tinggi. Jika neighbor terbaik tersebut memiliki nilai yang lebih besar daripada state saat ini, maka algoritma akan berpindah ke neighbor tersebut. Pencarian langsung berhenti begitu tidak ditemukan lagi neighbor dengan nilai yang lebih tinggi.

Algoritma ini bersifat greedy dan sangat efisien dalam penggunaan memori karena hanya menyimpan state saat ini tanpa perlu menyimpan pohon pencarian. Kelemahannya, algoritma ini tidak bersifat complete dan sangat rentan terjebak pada maksimum lokal (local maximum), dataran rata (plateau/shoulder), maupun punggungan (ridge) karena menolak perpindahan ke state dengan nilai yang sama atau lebih rendah. Akibatnya, kualitas solusi akhir yang dihasilkan sangat bergantung pada state awal.

1.1.2. Pseudocode

```
function STEEPEST-ASCENT-HILL-CLIMBING(problem) returns a
state that is a local maximum
  current <- problem.INITIAL-STATE
  loop do
    neighbor <- highest-valued of num_cand randomly selected
                successors of current
    if neighbor.VALUE <= current.VALUE then return current
    current <- neighbor
```

Bagian	Peran
`current <- problem.INITIAL-STATE`	Pencarian dimulai dari satu state awal yang diberikan
`loop do`	Perulangan utama yang terus berjalan selama masih ditemukan neighbor yang lebih baik
`highest-valued of num_cand ...`	Steepest ascent berbasis sampel, mengambil neighbor terbaik dari

`successors`	num_cand neighbor acak
`if neighbor.VALUE <= current.VALUE then return current`	Pencarian berhenti dan mengembalikan current ketika tidak ada neighbor yang nilainya lebih tinggi
`current <- neighbor`	Berpindah ke neighbor terbaik untuk melanjutkan iterasi berikutnya

1.1.3. Implementasi

Fungsi **steepest_ascent_hill_climbing** pada steepest_ascent.py digunakan untuk menjalankan satu kali pencarian steepest ascent dari state awal hingga berhenti di maksimum lokal.

```
import random
import time
from ...models.state import State
from ...utils.neighbor import random_neighbor

# Fungsi utama hill-climbing steepest ascent
def steepest_ascent_hill_climbing(initial_state: State,
num_cand: int = 50, seed=None) -> dict:
    rng = random.Random(seed)
    current = initial_state
    current.evaluate()

    value_history = [current.value]
    iterations = 0

    start = time.perf_counter()

    # Loop utama untuk iterasi steepest ascent
    while True:
        best = None
        best_value = current.value

        # Eksplorasi random neighbor sebanyak n dan mencari
        yang terbaik
        for _ in range(num_cand):
            cand = random_neighbor(current, rng)
            if cand is None:
                continue
            cand.evaluate()
            if cand.value > best_value:
                best = cand
                best_value = cand.value

        if best is None or best_value <= current.value:
            break

        current = best
        iterations += 1
        value_history.append(current.value)

    duration = time.perf_counter() - start
```

```
    # Mengembalikan format dictionary yang berisi data per
    state
    return {
        "initial_state": initial_state,
        "final_state": current,
        "final_value": current.value,
        "value_history": value_history,
        "iterations": iterations,
        "duration": duration,
    }
```

Parameter	Arti
`initial_state`	State awal tempat pencarian dimulai
`num_cand`	Jumlah neighbor acak yang dievaluasi pada tiap iterasi
`seed`	Seed generator acak agar hasil dapat direproduksi

Alur

1) Inisialisasi (generator acak rng dibuat dari seed, state awal ditetapkan sebagai current dan dievaluasi nilainya, serta mencatat nilai awal ke dalam *value_history* dan waktu mulai eksekusi).
2) Mengeksplorasi kandidat neighbor pada tiap iterasi dengan membangkitkan *num_cand* neighbor acak melalui *random_neighbor*, mengevaluasi nilainya, dan menyimpan kandidat dengan nilai tertinggi ke dalam best dan *best_value*.
3) Memeriksa kondisi berhenti, yaitu bila tidak ada kandidat yang valid (*best is None*) atau nilai kandidat terbaik tidak lebih besar dari nilai state saat ini (*best_value <= current.value*), maka pencarian berhenti (break).
4) Memperbarui state saat ini (*current = best*), menambah penghitung iterasi (iterations += 1), dan mencatat nilai terbaru ke dalam *value_history*.
5) Menghitung total waktu eksekusi (*duration*) setelah keluar dari loop.
6) Mengembalikan hasil pencarian dalam bentuk dictionary.

Output

Variabel	Arti
`initial_state`	State awal sebelum pencarian dimulai
`final_state`	State akhir yang dicapai saat algoritma berhenti
`final_value`	Nilai objective dari *final_state*
`value_history`	Riwayat nilai current dari state awal hingga iterasi terakhir

`iterations`	*Jumlah iterasi perbaikan yang berhasil dilakukan*
`duration`	Waktu eksekusi dalam detik

1.2. Sideways Move Hill Climbing

1.2.1. Pengertian

Steepest ascent hill climbing langsung berhenti ketika tidak menemukan neighbor dengan nilai yang lebih tinggi, sehingga gagal melewati area datar (plateau) yang nilainya sama dengan state saat ini. Sideways Move Hill Climbing mengatasinya dengan mengizinkan perpindahan ke neighbor yang memiliki nilai sama (sideways move) ketika tidak ditemukan neighbor yang bernilai lebih tinggi, dengan harapan area datar tersebut merupakan bahu (shoulder) yang mengarah ke lereng naik berikutnya.

Agar algoritma tidak terjebak di dalam infinite loop apabila area datar tersebut ternyata merupakan maksimum lokal datar (flat local maximum), jumlah perpindahan mendatar secara berturut-turut dibatasi hingga batas maksimum tertentu (max_side). Apabila algoritma berhasil menemukan neighbor dengan nilai yang lebih tinggi, maka penghitung sideways move akan di-reset kembali ke nol. Kelemahannya, algoritma ini membutuhkan iterasi tambahan saat menyusuri plateau dan tetap tidak dapat keluar dari lembah atau maksimum lokal yang dikelilingi oleh neighbor bernilai lebih rendah.

1.2.2. Pseudocode

```
function SIDEWAYS-MOVE-HILL-CLIMBING(problem, max_side)
returns a state
  current <- problem.INITIAL-STATE
  sideways_moves <- 0
  loop do
    neighbor <- highest-valued of num_cand randomly selected
                successors of current
    if neighbor.VALUE < current.VALUE then return current
    if neighbor.VALUE = current.VALUE then
      if sideways_moves >= max_side then return current
      sideways_moves <- sideways_moves + 1
    else
      sideways_moves <- 0
    current <- neighbor
```

Bagian	Peran
`sideways_moves <- 0`	Penghitung jumlah sideways move berturut-turut yang telah dilakukan
`highest-valued of num_cand ... successors`	Mengambil neighbor terbaik dari num_cand neighbor acak

`if neighbor.VALUE = current.VALUE then ...`	Pencarian berhenti bila seluruh neighbor yang dievaluasi bernilai lebih rendah dari state saat ini
`if neighbor.VALUE = current.VALUE then ...`	Mengizinkan langkah mendatar (sideways move) selama belum mencapai batas max_side
`if sideways_moves >= max_side then return current`	Mencegah infinite loop pada flat local maximum dengan menghentikan pencarian saat batas tercapai
`else sideways_moves <- 0`	Me-reset penghitung sideways move kembali ke 0 ketika berhasil menemukan langkah naik (uphill)

1.2.3. Implementasi

Fungsi **sideways_move_hill_climbing** pada sideways.py digunakan untuk menjalankan pencarian hill climbing yang mengizinkan langkah mendatar hingga batas max_side berturut-turut.

```
import random
import time
from ...models.state import State
from ...utils.neighbor import random_neighbor

# Fungsi utama untuk run sideways move HC
def sideways_move_hill_climbing(initial_state: State,
num_cand: int = 50, max_side: int = 100, seed=None) -> dict:
    rng = random.Random(seed)
    current = initial_state
    current.evaluate()

    value_history = [current.value]
    iterations = 0
    sideways_moves = 0

    start = time.perf_counter()

    # Outer loop untuk iterasi algoritma
    while True:
        best = None
        best_value = current.value

        # Inner loop untuk mencari n neighbor dan memilih
        yang terbaik
        for _ in range(num_cand):
            cand = random_neighbor(current, rng)
            if cand is None:
                continue
            cand.evaluate()
            if best is None or cand.value > best_value:
                best = cand
                best_value = cand.value
```

```
        if best is None or best_value < current.value:
            break

        # Iterasi terus menerus sampai mencapai batas max
        sideways
        if best_value == current.value:
            if sideways_moves >= max_side:
                break
            sideways_moves += 1
        else:
            sideways_moves = 0

        current = best
        iterations += 1
        value_history.append(current.value)

    duration = time.perf_counter() - start

    # Mengembalikan format dictionary yang berisi data per
    state
    return {
        "initial_state": initial_state,
        "final_state": current,
        "final_value": current.value,
        "value_history": value_history,
        "iterations": iterations,
        "sideways_moves": sideways_moves,
        "duration": duration,
    }
```

Parameter	Arti
`initial_state`	State awal tempat pencarian dimulai
`num_cand`	Jumlah neighbor acak yang dievaluasi pada tiap iterasi
`seed`	Seed generator acak agar hasil dapat direproduksi
`max_side`	Batas maksimum sideways move berturut-turut yang diperbolehkan

Alur

1) Inisialisasi (generator acak rng dibuat dari seed, state awal dievaluasi, serta menginisialisasi *value_history*, iterations, dan nilai *sideways_moves* = 0).
2) Mengeksplorasi kandidat neighbor pada tiap iterasi dengan membangkitkan *num_cand* neighbor acak dan memilih kandidat dengan nilai tertinggi sebagai *best* dan *best_value*.
3) Memeriksa kondisi berhenti utama, yaitu bila tidak ditemukan kandidat valid (*best is None*) atau kandidat terbaik bernilai lebih rendah dari state saat ini (*best_value* < *current.value*), maka pencarian berhenti.

4) Memeriksa kondisi sideways move, yaitu bila nilai kandidat terbaik sama dengan state saat ini (*best_value == current.value*), algoritma mengecek apakah *sideways_moves >= max_side*. Jika sudah mencapai batas, pencarian berhenti. Jika belum, *sideways_moves* ditambah 1. Sebaliknya, bila *best_value > current.value*, maka *sideways_moves* di-reset menjadi 0.
5) Memperbarui state saat ini (*current = best*), menambah jumlah iterasi (*iterations += 1*), dan mencatat nilai terbaru ke dalam *value_history*.
6) Menghitung durasi eksekusi (*duration*) dan mengembalikan hasil pencarian dalam bentuk dictionary.

Output

Variabel	Arti
`initial_state`	State awal sebelum pencarian dimulai
`final_state`	State akhir yang dicapai saat algoritma berhenti
`final_value`	Nilai objective dari *final_state*
`value_history`	Riwayat nilai current dari state awal hingga iterasi terakhir
`iterations`	Total iterasi perpindahan (baik naik maupun mendatar) yang dilakukan
`duration`	Waktu eksekusi dalam detik
`sideways_moves`	Jumlah *sideways move* berturut-turut pada saat algoritma berhenti

1.3. Random Restart Hill Climbing

1.3.1. Pengertian

Steepest ascent hill climbing berhenti begitu mencapai maksimum lokal sehingga hasilnya bergantung pada *state* awal. Random Restart Hill Climbing mengatasinya dengan menjalankan pencarian berkali-kali dari *state* awal acak yang berbeda, lalu mengembalikan hasil terbaik dari seluruh percobaan.

Algoritma ini bersifat *complete* secara probabilistik. Bila satu kali pencarian berhasil mencapai optimum global dengan peluang p, rata-rata dibutuhkan $\frac{1}{p}$ *restart* sampai berhasil sehingga dengan *restart* yang cukup banyak hampir pasti salah satunya menemukan solusi yang sangat baik. Kelemahannya, waktu bertambah sebanding dengan jumlah *restart*, dan tiap *restart* tidak memanfaatkan hasil restart sebelumnya.

1.3.2. Pseudocode

```
function RANDOM-RESTART-HILL-CLIMBING(problem, max_restart)
returns a solution state
  best <- null
  for r = 1 to max_restart do
    current <- RANDOM-INITIAL-STATE(problem)
    loop do
      next <- highest-valued of num_cand randomly selected
              successors of current
      if next.VALUE <= current.VALUE then break
      current <- next
    if best = null or current.VALUE > best.VALUE then
       best <- current
  return best
```

Bagian	Peran
`for r = 1 to max_restart`	Jumlah *restart* sekaligus kondisi algoritma berhenti
`RANDOM-INITIAL-STATE`	Setiap *restart* mulai dari *state* awal acak yang berbeda
`highest-valued of num_cand ... successors`	Steepest ascent berbasis sampel, mengambil *neighbor* terbaik dari num_cand *neighbor* acak
`if next.VALUE <= current.VALUE then break`	Sebuah *restart* selesai ketika tidak ada *neighbor* yang lebih baik
`best <- current`	Hasil *restart* disimpan hanya bila lebih baik dari yang terbaik sejauh ini

1.3.3. Implementasi

`Steepest_accent` untuk satu kali pencarian dan **`random_restart_hill_climbing`** sebagai *loop restart* di luarnya.

```
import random
import time
from ...models.state import State
from ...models.truck import Truck
from ...utils.neighbor import random_neighbor
from ...utils.state_utils import random_initial_state

# Fungsi untuk satu run steepest ascent
def steepest_ascent(current: State, rng: random.Random,
num_cand: int):
    current.evaluate()
    history = [current.value]
    iters = 0
    while True:
        best = None
        best_value = current.value
```

```
        for _ in range(num_cand):
            cand = random_neighbor(current, rng)
            if cand is None:
                continue
            cand.evaluate()
            if cand.value > best_value:
                best = cand
                best_value = cand.value
        if best is None or best_value <= current.value:
            break
        current = best
        iters += 1
        history.append(current.value)
    return current, iters, history

# Fungsi utama untuk algirtma random restart
def random_restart_hill_climbing(packages, truck: Truck,
max_restart: int = 10, num_cand: int = 50, seed=None) ->
dict:
    rng = random.Random(seed)
    start = time.perf_counter()

    best_overall = None
    first_state = None
    iterations_per_restart = []
    all_histories = []
    best_history = []
    total_iters = 0

    # Outer loop sebagai maksimum random restart
    for _ in range(max_restart):
        restart_seed = rng.randint(0, 2**31 - 1)
        init = random_initial_state(packages, truck,
               seed=restart_seed)
        if first_state is None:
            first_state = init

        # Memanggil inner loop untuk iterasi
        final, iters, history = steepest_ascent(init, rng,
        num_cand)

        iterations_per_restart.append(iters)
        all_histories.append(history)
        total_iters += iters

        if best_overall is None or final.value >
        best_overall.value:
            best_overall = final
        best_history.append(best_overall.value)

    duration = time.perf_counter() - start

    # Mengembalikan format dictionary yang berisi data per
      state
    return {
        "initial_state": first_state,
        "final_state": best_overall,
        "final_value": best_overall.value if best_overall is
```

```
                        not None else float("-inf"),
        "value_history": best_history,
        "all_histories": all_histories,
        "iterations": total_iters,
        "iterations_per_restart": iterations_per_restart,
        "total_restarts": max_restart,
        "duration": duration,
    }
```

Parameter	Arti
`packages`	Daftar paket yang akan dimuat
`truck`	Truk (*ruang dan kapasitas*)
`max_restart`	Jumlah *restart*
`num_cand`	Jumlah *neighbor* acak yang dievaluasi pada tiap iterasi
`seed`	Seed generator acak agar hasil dapat direproduksi

Alur

7) Inisialisasi (*generator acak dibuat dari seed*)
8) Membangkitkan *state* awal dengan tiap *restart*, *rng* menghasilkan *seed* baru dan *random_initial_state* membentuk *state* awal dari *seed* tersebut. *State* awal *restart* pertama disimpan sebagai *first_state*
9) Menjalankan steepest accent yang mengevaluasi *state* awal, lalu pada tiap iterasi membangkitkan *num_cand neighbor* acak dan mengingat yang nilainya paling tinggi. Bila ada *neighbor* yang lebih baik dari *state* saat ini, algoritma pindah ke sana dan mencatat nilainya. Bila tidak ada, pencarian *restart* ini berhenti.
10) Mencatat hasil *restart*
11) Memperbarui hasil terbaik
12) Mengembalikkan hasil

Output

Variabel	Arti
`initial_state`	State awal *restart* pertama
`final_state`	State terakhir dari seluruh *restart*
`final_value`	Nilai *final_state*
`value_history`	Nilai terbaik sejauh ini setelah tiap *restart*
`all_histories`	Riwayat nilai dari setiap *restart*
`iterations`	Total iterasi perbaikan di seluruh *restart*
`iterations_per_rest`	Jumlah iterasi perbaikan pada tiap *restart*

`art`	
`total_restarts`	Jumlah *restart* yang dijalankan
`duration`	Waktu eksekusi dalam detik

1.4. Stochastic Hill Climbing

1.4.1. Pengertian

Stochastic Hill Climbing memilih satu *neighbor* secara acak pada tiap iterasi, lalu berpindah hanya bila *neighbor* itu lebih baik dari *state* saat ini. Berbeda dari steepest ascent yang menilai banyak *neighbor* sebelum bergerak, algoritma ini hanya mengevaluasi satu *neighbor* per iterasi sehingga biaya per iterasinya rendah.

Akibatnya, penyelesaiannya lebih lambat dan hasil bergantung pada *seed* acak karena arah perbaikan ditemukan secara kebetulan. Algoritma ini juga tetap dapat terjebak di maksimum lokal. Algoritma ini tidak pernah menerima langkah yang lebih buruk.

1.4.2. Pseudocode

```
function STOCHASTIC-HILL-CLIMBING(problem, nmax) returns a
solution state
  current <- MAKE-NODE(problem.INITIAL-STATE)
  for t = 1 to nmax do
    next <- a randomly selected successor of current
    if next.VALUE > current.VALUE then current <- next
  return current
```

Bagian	Peran
`for t = 1 to nmax`	Jumlah percobaan dibatasi *nmax*
`next <- a randomly selected successor`	Hanya satu *neighbor* acak yang dibangkitkan dan dievaluasi
`if next.VALUE > current.VALUE`	*State* berpindah hanya ketika *neighbor* lebih baik

1.4.3. Implementasi

```
import random
import time
from ...models.state import State
from ...utils.neighbor import random_neighbor

# Fungsi utama untuk menjalankan algoritma stochastic HC
def stochastic_hill_climbing(initial_state: State, nmax: int
= 1000, seed=None) -> dict:
    rng = random.Random(seed)
```

```
    current = initial_state
    current.evaluate()

    value_history = [current.value]

    start = time.perf_counter()

    # Iterasi stochastic HC sebanyak n kali
    for _ in range(nmax):
        cand = random_neighbor(current, rng)
        if cand is None:
            value_history.append(current.value)
            continue
        cand.evaluate()
        if cand.value > current.value:
            current = cand
        value_history.append(current.value)

    duration = time.perf_counter() - start

    # Mengembalikan format dictionary yang berisi data per
    state
    return {
        "initial_state": initial_state,
        "final_state": current,
        "final_value": current.value,
        "value_history": value_history,
        "iterations": nmax,
        "duration": duration,
    }
```

Parameter	Arti
`initial_state`	*State* awal
`nmax`	Jumlah percobaan *neighbour* (iterasi) (*default 1000*)
`seed`	Seed generator acak agar hasil dapat direproduksi

Alur iterasi

1) Membangkitkan *neighbour* dengan mengambil satu *neighbour* acak
2) Mengevaluasi *neighbour*
3) Membandingkan apabila *cand.value > current.value*. Jika lebih baik, *state* pindah ke *neighbour*. Bila tidak, *state* tetap di *current*
4) Mencatatnya ke *value_history*
5) Setelah *nmax* iterasi, algoritma mengembalikan *state* terakhir beserta riwayat nilainya\

Output

Variabel	Arti

`initial_state`	State awal
`final_state`	State terakhir (*current*)
`final_value`	Nilai *final_state*
`value_history`	Nilai *current* sebelum iterasi pertama dan setelah tiap iterasi
`all_histories`	Riwayat nilai dari setiap *restart*
`iterations`	Jumlah percobaan
`duration`	Waktu eksekusi dalam detik

2. Simulated Annealing Algorithm

2.1. Pengertian

Simulated Annealing (SA) menggabungkan 2 pendekatan yang saling melengkapi. *Random walk* bersifat *complete* karena pada akhirnya dapat menjangkau seluruh state, tetapi tidak efisien karena mengabaikan nilai fungsi objektif. Hill climbing efisien karena selalu bergerak ke arah yang lebih baik, tetapi *incomplete* karena berhenti di maksimum lokal. SA mengambil keduanya, yaitu melakukan pencarian yang tetap condong ke perbaikan, tetapi sesekali menerima langkah yang lebih buruk agar tidak terjebak.

Nama algoritma berasal dari *annealing* pada metalurgi, yaitu logam yang dipanaskan lalu didinginkan perlahan sampai atomnya menempati susunan yang stabil. Analoginya, nilai objektif berperan sebagai energi dan temperatur T mengatur seberapa bebas pencarian bergerak.

Untuk $\Delta E \leq 0$, nilai $e^{\frac{\Delta E}{T}}$ berada di (0, 1] dan bergantung pada dua hal:

- Semakin buruk langkahnya ($|\Delta E|$ besar), peluang penerimaan semakin kecil
- Semakin rendah temperatur, peluang penerimaan semakin kecil

Akibatnya, pada T tinggi, algoritma mengeksplorasi secara luas karena langkah buruk mudah untuk diterima, sedangkan pada T rendah, algoritma berperilaku seperti hill climbing. Bila T diturunkan cukup lambat, SA secara teoritis akan menemukan optimum global dengan peluang mendekati 1.

2.2. Pseudocode

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution
state
  current <- MAKE-NODE(problem.INITIAL-STATE)
  for t = 1 to infinity do
    T <- schedule(t)
    if T = 0 then return current
    next <- a randomly selected successor of current
    dE <- next.VALUE - current.VALUE
    if dE > 0 then current <- next
```

```
    else current <- next only with probability e^(dE/T)
```

Bagian	Peran
`schedule(T)`	Fungsi yang memetakan waktu ke temperatur (*T menurun seiring waktu*)
`If T = 0 then return current`	Pencarian selesai ketika T mendekati 0 (*sistem dingin*)
`next <- a randomly selected successor`	Hanya satu neighbor acak yang dibangkitkan
`dE > 0 then current <- next`	Langkah yang lebih baik selalu diterima (*stochastic hill climbing*)
`else ... probability` $e^{\frac{\Delta E}{T}}$	Langkah yang lebih buruk diterima dengan peluang tertentu

2.3. Implementasi

```
def simulated_annealing(init, rng, t_start=100.0, alpha=0.95,
t_min=1e-2) -> dict:
    init.evaluate()
    curr = init
    best = init
    history, probs = [], []
    stuck = 0
    start = time.time()

    turn = 1
    while True:
        temp = schedule(turn, t_start, alpha)
        if temp < t_min:
            break

        neighbor = random_neighbor(curr, rng)
        p = 0.0
        moved = False
        if neighbor is not None:
            neighbor.evaluate()
            if not math.isinf(neighbor.value):
                delta = neighbor.value - curr.value
                if delta > 0:
                    p = 1.0
                    curr = neighbor
                    moved = True
                else:
                    p = math.exp(delta / temp)

                    # kalau random < p, pindah ke neighbor
                    if rng.random() < p:
                        curr = neighbor
                        moved = True
```

```
                if curr.value > best.value:
                    best = curr

        if not moved: # kalau curr tidak pindah, stuck bertambah 1
            stuck += 1

        history.append(curr.value)
        probs.append(p)
        turn += 1

    return {
        "initial": init,
        "final": curr,
        "best": best,
        "history": history,
        "probs": probs,
        "stuck": stuck,
        "duration": time.time() - start,
    }
```

Parameter	Arti
`init`	State awal, yaitu seluruh paket berada di luar truk
`rng`	Generator bilangan acak dengan *seed* agar hasil dapat direproduksi
`t_start`	Temperatur awal (*default 100.0*)
`t_min`	Kondisi berhenti (*default 0.01*)
`alpha`	Faktor pendingin temperatur (*default 0.95*)

Fungsi Jadwal Pendinginan

`schedule(turn, t_start, alpha)` memetakan nomor iterasi ke temperatur dengan rumus pendinginan:

$$T(t) = t_{start} . \alpha^{t-1}$$

Pada iterasi pertama (t = 1), nilai temperatur awal ditentukan menggunakan t_{start}, kemudian nilai T dikalikan dengan α pada setiap iterasi berikutnya.

Alur Iterasi

1) Hitung temperatur (`temp = schedule(turn, t_start, alpha)`). Variabel *turn* dimulai dari 1, sehingga iterasi pertama memakai t_{start}
2) Mengecek kondisi berhenti (*Jika temp < t_min, loop dihentikan*)
3) Bangkitkan *neighbour* yang akan mengambil satu neighbor acak. Bila tidak ada neighbor yang dapat dibentuk, iterasi dihitung mengalami *stuck*
4) Melakukan evaluasi (*apabila neighbour bernilai inf, neighbour dibuang*)
5) Hitung selisih ($\Delta E = neighbour.value - current.value$)

6) Menentukan apakah state akan berpindah atau tidak. Jika $\Delta E > 0$, algoritma akan pindah ke *neighbour* dengan p = 1.0. Bila tidak, p = $e^{\frac{\Delta E}{T}}$ dan algoritma akan pindah ke *neighbour* jika *rng.random()* < p
7) Perbarui *best*. Bila *curr* > dari *best*, *best* diganti
8) Catat dan lanjut. Nilai *curr* dan p dicatat, lalu turn bertambah 1

Output

Variabel	Arti
`initial`	State awal
`final`	State terakhir (*curr*)
`best`	State terbaik
`history`	Nilai *curr* untuk setiap iterasi
`probs`	Peluang penerimaan untuk setiap iterasi
`stuck`	Jumlah iterasi tanpa perpindahan
`duration`	Waktu eksekusi dalam detik

3. Genetic Algorithm

3.1. Pengertian Genetic Algorithm

Genetic Algorithm melalui beberapa proses, yaitu Selection, Crossover, dan Mutation. Tujuannya untuk menemukan kombinasi state lain yang berkemungkinan lebih baik dari sebelumnya. Sama seperti algoritma yang lain, Genetic Algorithm juga bertujuan untuk mencari global optimum walau pada prosesnya, seringkali dibatasi oleh waktu komputasi sehingga tidak selalu menemukan global optimum terlebih pada program ini.

Sebagai catatan, Genetic Algorithm di program ini menggunakan heuristic tambahan, yaitu menyimpan state terbaik dari tiap iterasi dan terus memasukkan state terbaik ke dalam iterasi selanjutnya. State terbaik akan menggantikan state terburuk dari populasi di iterasi tersebut. Harapannya untuk mengarahkan algoritma untuk menuju ke global optimum dengan terus memasukkan state terbaik.

3.2. Alur Program dan Pseudocode

Alur program dari Genetic Algorithm adalah sebagai berikut.

A. Persiapan

Ada beberapa fungsi yang dibuat untuk mewujudkan genetic algorithm. Fungsi tersebut mencakup fitness function dan objective statistics. Fitness function merupakan fungsi untuk menghitung fitness scores dari suatu state. Fitness scores dibedakam dengan objective value karena nilai minus (artinya tidak valid) diperbolehkan di dalam fitness scores. Fitness scores juga dilengkapi dengan perhitungan penalty melalui fungsi calculate_penalty(). Fitness function dan fitneas scores akan dipakai untuk

menentukan state terbaik dan probabilitas state dipilih di dalam bagian selection.

Untuk fungsi objective statistics, kegunannya adalah untuk menghitung data statistik dari tiap iterasi untuk mencari nilai maksimum dan nilai rata-rata dari objective value seluruh state di dalam populasi agar hasil dari algoritma sesuai dengan yang diharapkan (di dalam spesifikasi)

B. Selection

Selection menggunakan roulette wheel yang menggunakan fitness scores sebagai probabilitas. Lalu, tiap iterasi, akan dihasilkan sebuah angka random 0-1 untuk menentukan parent yang akan dipilih dari sebuah population. Jumlah parent yang dipilih akan sama dengan jumlah populasi. Parent-parent ini nantinya akan melalui crossover dan mutation.

C. Crossover

Crossover di dalam masalah ini akan menukar beberapa paket di antara dua parent. Paket yang ditukar merupakan paket yang sama dan yang berbeda hanyalah konfigurasi, seperti masuk/tidaknya ke dalam truk, dan posisi di dalam truk. Proses crossover akan meng-iterasi semua parent yang ada dengan memasangkan dua parent secara berurutan. Pada setiap pasangan parent, akan dipilih satu titik potong (*crossover point*) secara random untuk menukar daftar paket dan menghasilkan dua child. Setiap child yang terbentuk kemudian divalidasi menggunakan fungsi evaluate(). Apabila child yang dihasilkan tidak valid (melanggar aturan), maka child tersebut akan dikembalikan menjadi salinan dari parent asalnya. Jika jumlah populasi bernilai ganjil, maka satu parent terakhir yang tidak memiliki pasangan akan langsung disalin ke populasi baru.

D. Mutation

Mutation dilakukan untuk memberikan variasi acak pada setiap individual (state) di dalam populasi baru hasil crossover agar algoritma tidak terjebak pada *local optimum*.

Pada tahap ini, setiap state akan dicek menggunakan sebuah angka random 0–1 terhadap nilai probabilitas mutation. Apabila angka random yang dihasilkan lebih kecil dari nilai probabilitas tersebut, maka state tersebut akan mengalami mutasi dengan cara mencari random neighbor (melalui operasi move, rotate, atau swap paket). Jika dari percobaan tersebut ditemukan state baru yang valid, maka state hasil mutasi akan menggantikan state sebelumnya. Namun, jika tidak ditemukan hasil mutasi yang valid hingga batas percobaan habis, maka state awal akan tetap dipertahankan.

E. Hasil Akhir Tiap Iterasi

Setelah populasi baru selesai melewati tahap selection, crossover, dan mutation, program akan menghitung kembali fitness scores dari seluruh state di dalam populasi baru tersebut. Selanjutnya, dilakukan proses pemasukan state terbaik ke populasi baru untuk menjaga agar state terbaik dari generasi sebelumnya tidak hilang. Proses ini bekerja dengan mencari state yang memiliki fitness score terburuk di dalam populasi baru, lalu menggantinya dengan individu terbaik dari iterasi sebelumnya apabila memiliki fitness score yang lebih tinggi.

Setelah itu, populasi baru beserta fitness scores-nya akan ditetapkan menjadi populasi aktif untuk iterasi berikutnya. Di akhir tiap iterasi,

program akan mencari state dengan objective value tertinggi pada generasi tersebut. Jika nilainya lebih besar daripada semua *objective value* sejauh ini, maka state tersebut akan dibuat menjadi state terbaik dan akan menjadi state yang dibandingkan untuk iterasi selanjutnya.

F. Main Genetic Algorithm

Fungsi utama Genetic Algorithm terdiri dari dua bagian, yaitu fungsi pertama sebagai pengatur jalannya algoritma dengan yang akan memanggil fungsi kedua, fungsi yang menjalankan satu siklus Genetic Algorithm. Hal ini diperlukan karena algoritma akan dijalankan sebanyak total 9 kali.

Pada fungsi pertama (genetic_algorithm), program menerima pilihan variabel kontrol berupa "jumlah_populasi" atau "banyak_iterasi". Variabel yang tidak dikontrol akan divariasikan secara random sebanyak 3 nilai berbeda (dalam rentang 1–100), dan setiap variasi akan dijalankan sebanyak 3 kali melalui fungsi kedua (run_genetic_algorithm) sehingga menghasilkan total 9 percobaan. Sementara itu, fungsi kedua akan mengeksekusi tahapan persiapan, melakukan iterasi sebanyak jumlah iterasi (yang berisi selection, crossover, mutation, dan pencatatan statistik), serta mengembalikan hasil akhir beserta waktu eksekusinya.

Berikut adalah pseudocode dari fungsi utama Genetic Algorithm:

```
// Fungsi Pertama
FUNCTION genetic_algorithm(packages, truck, pilihan, n)
     init_population()
     tentukan jumlah iterations berdasarkan pilihan dan n
     Jalankan run_genetic_algorithm(population, iterations) untuk
setiap variasi percobaan
    RETURN results
END FUNCTION

// Fungsi Kedua
FUNCTION run_genetic_algorithm(population, iterations)
    fitness_scores <- [fitness_function(state) FOR state IN
population]
    //state terbaik di population
    initial_state, best_state <- copy_state
    Simpan objective_statistics(population) ke history

    FOR step <- 1 TO iterations DO
        parents <- selection(population, fitness_scores)
        new_population <- crossover(parents)
        new_population <- [mutation(state) FOR state IN
new_population]
        new_fitness <- [fitness_function(state) FOR state IN
new_population]
        # State Terbaik
        Ganti state terburuk di new_population dengan
copy_state(best_state) jika best_state lebih baik
        population, fitness_scores <- new_population, new_fitness
        Perbarui best_state jika ada state di population dengan
objective_value() lebih tinggi
        Simpan objective_statistics(population) ke history
    END FOR
    best_state.evaluate()
```

```
    RETURN initial_state, best_state, history, duration
END FUNCTION
```

3.3. Implementasi

Implementasi kode untuk Genetic Algorithm adalah seabgai berikut.

```
// Fungsi Pertama
def genetic_algorithm(packages, truck, pilihan, n, seed=None) -> dict[str, object]:
    rng = random.Random(seed)
    results = []

    if pilihan == "jumlah_populasi":
        jumlah_populasi = n
        if jumlah_populasi <= 0:
            raise ValueError("Jumlah populasi harus lebih besar dari nol")

        population = init_population(jumlah_populasi, packages, truck, rng=rng)

        for _ in range(3):
            iteration = rng.randint(1, 100)
            for _ in range(3):
                results.append(run_genetic_algorithm(population, iteration,
rng=rng))
    else:
        iteration = n
        if iteration < 0:
            raise ValueError("Banyak iterasi tidak boleh negatif")

        for _ in range(3):
            jumlah_populasi = rng.randint(1, 100)
            population = init_population(jumlah_populasi, packages, truck, rng=rng)
            for _ in range(3):
                results.append(run_genetic_algorithm(population, iteration,
rng=rng))

    return {
        "pilihan": pilihan,
        "nilai_kontrol": n,
        "results": results,
    }

// Fungsi Kedua
def run_genetic_algorithm(population: list[State], iterations: int, rng=random) ->
dict[str, object]:

    # 1. Menghitung wkatu mulai, ukuran populasi, dan skor fitness awal
    start = perf_counter()
    population_size = len(population)
    fitness_scores = [fitness_function(state) for state in population]

    # 2. Menentukan state awal dan sate terbaik beserta valuenya
    initial_best = population[0]
    initial_best_value = initial_best.objective_value()
    for state in population:
        value = state.objective_value()
        if value > initial_best_value:
            initial_best = state
            initial_best_value = value
    initial_state = copy_state(initial_best)
    best_state = copy_state(initial_best)
    best_objective = best_state.objective_value()

    # 3. Menyimpan nilai maksimum dan rata-rata value dari populasi awal
    maxobjective, avgobjective = objective_statistics(population)
    maxhistory = [maxobjective]
    avghistory = [avgobjective]
```

```
    for _ in range(iterations):
        # I. Selection
        parents = selection(population, fitness_scores, rng=rng)
        new_population = []

        # II. Crossover
        for index in range(0, population_size - 1, 2):
            child1, child2 = crossover(
                parents[index],
                parents[index + 1],
                rng=rng,
            )
            new_population.extend((child1, child2))

        if population_size % 2 == 1:
            new_population.append(copy_state(parents[-1]))

        # III. Mutation
        new_population = [mutation(individual, rng=rng) for individual in
        new_population]

        # Menghitung skor fitness dari populasi baru
        new_population_fitness = [fitness_function(state) for state in
        new_population]

        # Elitism (menyimpan state terbaik dari populasi sebelumnya)
        worst_index = 0
        worst_value = new_population_fitness[0]
        for index in range(population_size):
            if new_population_fitness[index] < worst_value:
                worst_index = index
                worst_value = new_population_fitness[index]
        best_fitness = fitness_function(best_state)
        if best_fitness > new_population_fitness[worst_index]:
            new_population[worst_index] = copy_state(best_state)
            new_population_fitness[worst_index] = best_fitness

        # Hasil Akhir Proses Iterasi
        population = new_population
        fitness_scores = new_population_fitness

        # [Informasi Statistik]
        generation_best = population[0]
        generation_best_objective = generation_best.objective_value()
        for state in population:
            value = state.objective_value()
            if value > generation_best_objective:
                generation_best = state
                generation_best_objective = value

        if generation_best_objective > best_objective:
            best_state = copy_state(generation_best)
            best_objective = generation_best_objective

        maxobjective, avgobjective = objective_statistics(population)
        maxhistory.append(maxobjective)
        avghistory.append(avgobjective)

    # Hasil Akhir
    best_state.evaluate()
    duration = perf_counter() - start

    return {
        "initial_state": initial_state,
        "final_state": best_state,
        "final_objective": best_objective,
        "objective_history": {
            "iteration": list(range(iterations + 1)),
            "maximum": maxhistory,
            "average": avghistory,
        },
        "population_size": population_size,
```

```
        "iterations": iterations,
        "duration_seconds": duration,
    }
```

A. Fungsi Pertama (genetic_algorithm())

Fungsi genetic_algorithm merupakan fungsi utama yang mengatur jalannya Genetic Algorithm berdasarkan variabel kontrol yang dipilih pengguna ("jumlah_populasi" atau "banyak_iterasi"), lalu memanggil run_genetic_algorithm untuk setiap percobaan.

Tabel Parameter Input dan Output genetic_algorithm:

Input

Parameter	Arti
`packages`	Daftar seluruh paket yang akan dimasukkan ke dalam truk.
`truck`	Objek truk yang menyimpan dimensi kontainer dan kapasitas berat maksimum.
`pilihan`	Mode variabel kontrol ("jumlah_populasi" atau "banyak_iterasi").
`n`	Nilai dari variabel kontrol sesuai dengan pilihan yang ditentukan.

Output

Parameter	Arti
`pilihan`	Mode variabel kontrol ("jumlah_populasi" atau "banyak_iterasi").
`results`	List of “hasil dari setiap iterasi algoritma yang dijalankan fungsi run_genetic_algorithm”

B. Fungsi Kedua (run_genetic_algorithm())

Fungsi run_genetic_algorithm merupakan fungsi inti yang mengeksekusi satu kali proses pencarian Genetic Algorithm pada suatu populasi awal sebanyak jumlah iterasi yang diberikan.

Tabel Parameter Input dan Output run_genetic_algorithm:

Input

Parameter	Arti
`population`	Daftar state pada populasi awal yang akan diproses.
`iterations`	List of “hasil dari setiap iterasi algoritma yang dijalankan fungsi run_genetic_algorithm”

Output

Parameter	Arti
`initial_state`	State terbaik dari populasi awal sebelum iterasi dimulai.
`final_state`	State terbaik yang ditemukan setelah seluruh iterasi selesai.
`final_objective`	Nilai objective function tertinggi dari "final_state".
`objective_history`	Riwayat statistik tiap iterasi yang berisi daftar "iteration", "maximum" (maxhistory), dan "average" (avghistory).
`population_size`	Ukuran populasi yang digunakan di dalam proses pencarian.
`iterations`	Jumlah iterasi yang dijalankan.
`duration_seconds`	Durasi waktu eksekusi algoritma dalam satuan detik.

C. Fungsi Tahapan Genetic Algorithm (helper_ga.py)

Sebagai pendukung fungsi utama di atas, tahapan-tahapan utama Genetic Algorithm diimplementasikan ke dalam fungsi-fungsi berikut:

Fungsi	Tujuan
`init_population`	Membuat populasi awal sebanyak n state secara acak.
`fitness_function`	Menghitung fitness score suatu state (objective_value - penalty).
`selection`	Memilih daftar parent berukuran sama dengan populasi menggunakan roulette wheel selection.
`crossover`	Menukar konfigurasi paket dari dua parent pada satu titik potong acak untuk menghasilkan child1 dan child2.
`mutation`	Memutasi state berdasarkan peluang dengan mencari random neighbor yang valid.
`objective_statistics`	Mengembalikan pasangan nilai (max_objective, avg_objective) dari sebuah populasi.

4. Objective Function

4.1. Objective Function 1 (Wajib)

Objective function pada program ini bertujuan untuk memaksimumkan total nilai (*value*) dari seluruh paket yang berhasil dimuat ke dalam truk dengan tetap memenuhi seluruh batasan (*constraints*) yang berlaku. Secara matematis, objective value dari suatu state S dapat dihitung sebagai berikut.

$$Objective(S) = \Sigma value(p),\ untuk\ p\ yang\ berada\ di\ dalam\ truk$$

Pada implementasi fungsi *evaluate()* di **state.py**, nilai akhir dari suatu state ditentukan berdasarkan kevalidannya (*is_feasible(state*)):

1. Jika state tersebut valid, maka objective value dari state tersebut adalah total nilai package di dalam truk.
2. Jika state tersebut tidak valid, maka objective value adalah *-infinite*.

Pada implementasi Genetic Algorithm, Objective Function dipakai untuk menghitung fitness function/scores.

4.2. Objective Function 2 (Bonus)

Objective function kedua bertujuan untuk memaksimalkan total nilai paket yang berhasil dimuat ke dalam truk dengan mempertimbangkan kesesuaian urutan *Estimated Time Arrival* (ETA) antar paket. Objektif ini bekerja dengan cara memberikan penalti bagi paket dengan ETA yang lebih genting, apabila ditempatkan tepat di bawah paket dengan ETA yang kegentingannya lebih rendah. Objektif ini hanya akan menghitung paket yang berada di dalam truk. Secara matematis, fungsi objektif ini dituliskan sebagai berikut:

$$f(S) = V(S) - P(S)$$

- V(S): total nilai seluruh paket yang berada di dalam truk
- P(S) total penalti berdasarkan pelanggaran urutan ETA antarpaket

$$P(S) = max(100\Delta ETA,\ 100) \times k!$$

Delta ETA merupakan jumlah selisih ETA dari seluruh pasangan yang melanggar aturan, sedangkan *k* merupakan jumlah pasangan yang melanggar urutan ETA. Apabila tidak ditemukan pelanggaran, implementasi tetap menghasilkan penalti minimum, yakni sebesar 100.

5. Multitruk (Bonus)

5.1. Penjelasan Alur

Fitur multi-truk memungkinkan proses pemuatan paket dilakukan ke dalam beberapa kontainer truk. Pemrosesan dilakukan secara sekuensial yang berarti bahwa algoritma akan menyelesaikan satu truk baru lanjut ke truk selanjutnya. Implementasi tetap mempertahankan representasi satu *state* untuk satu truk.

Pada awal pencarian, seluruh daftar paket diberikan kepada truk 1 untuk dioptimasi menggunakan algoritma *local search* yang dipilih. Setelah algoritma pada truk 1 mencapai state terminal, paket-paket yang berhasil

dimuat akan dikunci pada truk 1, sedangkan sisa paket yang masih berada di luar dikumpulkan kembali menjadi *list* paket baru.

Selanjutnya, daftar paket yang belum terangkut tersebut diteruskan ke truk 2 untuk dijalankan kembali dengan algoritma yang sama pada dimensi dan kapasitas kontainer truk 2. Proses iterasi antar-truk ini terus berlanjut secara berurutan hingga salah satu kondisi berhenti terpenuhi, yaitu seluruh kontainer truk yang tersedia telah digunakan (truk habis) atau seluruh paket telah berhasil dimuat ke dalam truk (paket habis). Hasil akhirnya merupakan gabungan dari *state* masing-masing truk beserta akumulasi nilai fungsi objektif, beban muatan, durasi, dan jumlah iterasi. Secara umum, pendekatan yang kami gunakan adalah ***greedy* dengan *local search***.

5.2. Implementasi

Implementasi fitur multitruk dibagi ke dalam tiga bagian utama tanpa mengubah algoritma pencarian satu truk:

1. Parser Input Multitruk
   Fungsi **parse_trucks()** membaca satu atau beberapa baris spesifikasi kontainer (<W> <L> <H> <capacity>) secara berurutan pada berkas input .txt sebelum baris jumlah paket atau parameter algoritma, lalu menyimpannya sebagai list of truck.
2. Iterasi Truk
   Fungsi **solve()** melakukan iterasi untuk setiap truk dan menjalankan algoritma yang terpisah untuk masing-masing truk. Pemrosesan dijalankan secara sekuensial pada masing-masing truk dan paket akan disesuaikan dengan kondisi dari iterasi sebelumnya (bisa berkurang karena sudah di-assign ke truk sebelumnya)
3. Backend Multitruk untuk GUI
   Hasil akhir dari kasus multitruk adalah adanya *list* of *state* yang berarti ada banyak *state* dengan kondisi paket dan truk yang berkemungkinan berbeda. Nantinya, tiap *state* akan dipetakan ke nomor indeks truk untuk mendukung informasi lokasi package dan visualisasi 3d.

BAB III

ANALISIS DAN PEMBAHASAN

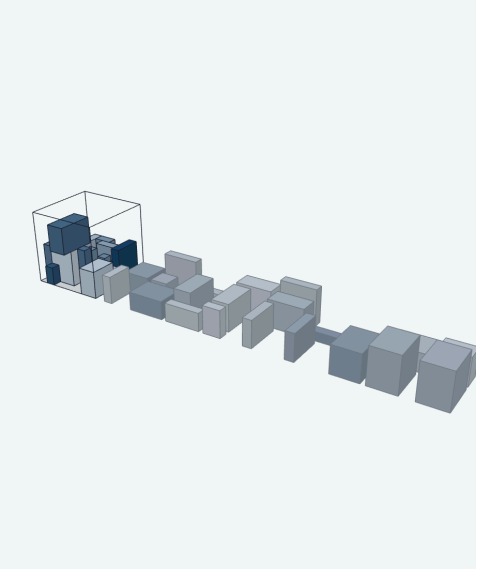
Test Case yang Digunakan (Seed = 42)

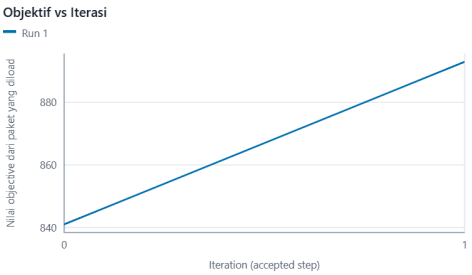
```
<Algoritma> (HC_STEEPREST/HC_SIDEWAYS/HC_STOCHASTIC/HC_RESTART/SA/GA)
8 8 8 109
jumlah_populasi | banyak_iterasi (GA)
10 (GA)
40
2 3 1 6 43 0 2
1 3 1 6 48 1 5
3 1 3 5 12 0 2
3 2 3 10 79 0 2
4 2 3 4 89 0 1
2 1 1 7 51 0 1
2 2 2 3 48 0 3
3 2 3 4 8 1 5
1 4 4 4 26 0 0
2 4 1 3 11 1 4
4 2 3 10 45 0 1
1 1 1 4 11 0 0
1 3 1 8 20 0 2
1 2 3 4 2 0 4
2 1 2 4 18 0 5
2 2 1 5 91 1 4
2 4 3 8 12 0 0
4 3 2 1 3 0 2
3 4 1 4 19 0 0
3 4 1 4 24 0 2
2 3 4 5 3 0 1
3 2 4 7 45 0 1
1 4 4 7 24 0 4
1 4 3 10 70 1 4
1 1 4 4 97 0 3
4 1 1 10 68 1 1
4 3 3 4 88 0 1
3 4 4 3 54 0 1
4 3 1 5 34 0 3
2 4 3 7 33 0 0
3 1 3 1 32 0 4
1 1 2 9 84 0 2
2 3 1 4 52 0 4
4 1 3 10 96 0 3
1 1 3 5 76 0 5
2 3 4 5 72 0 2
4 4 3 4 50 0 2
1 1 4 5 62 1 2
1 2 1 7 25 0 2
3 3 2 3 3 0 2
50 (HC) / 100 0.95 0.1 (SA)
```

1. Hill-Climbing Algorithm

a. Steepest Ascent HC (SAHC)

RUN 1





Hill climbing: steepest

Objective	Nilai total
Parameters	num_cand: 50
Base seed	42
Source	inline
Runs	1

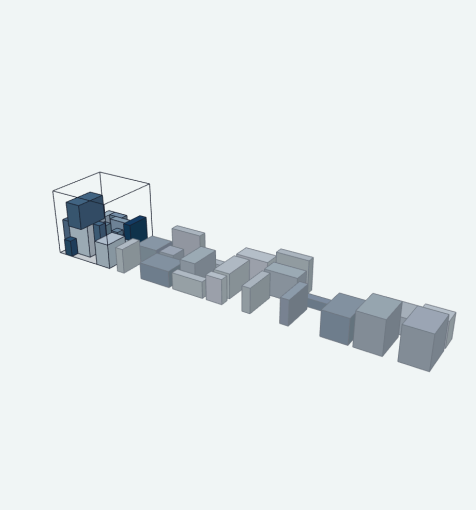
Run	Seed	Final objective	Iterations	Search duration (s)
Run 1	42	893	1	0.0547

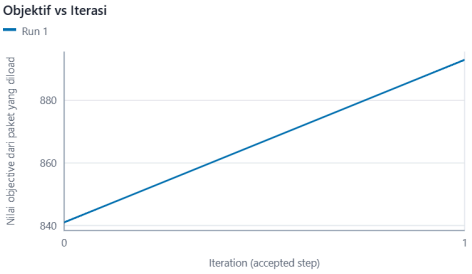
▶ Initial state: Initial state (random)

▶ Final state: Final state (terminal)

Final objective value	893
Search duration (s)	0.0547
Banyak iterasi hingga terminasi	1

RUN 2





Hill climbing: steepest

Objective	Nilai total
Parameters	num_cand: 50
Base seed	42
Source	inline
Runs	1

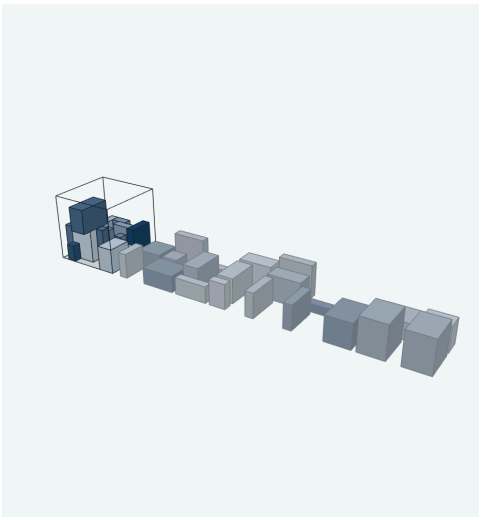
Run	Seed	Final objective	Iterations	Search duration (s)
Run 1	42	893	1	0.0548

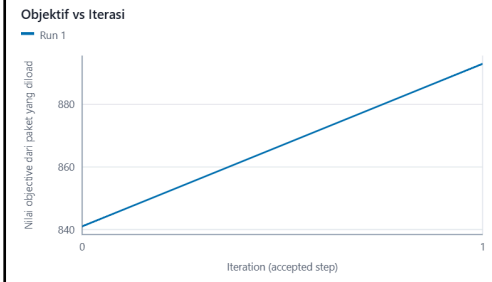
▶ Initial state: Initial state (random)

▶ Final state: Final state (terminal)

Final objective value	893
Search duration (s)	0.0548
Banyak iterasi hingga terminasi	1

RUN 3





Hill climbing: steepest

Objective	Nilai total
Parameters	num_cand: 50
Base seed	42
Source	inline
Runs	1

Run	Seed	Final objective	Iterations	Search duration (s)
Run 1	42	893	1	0.0543

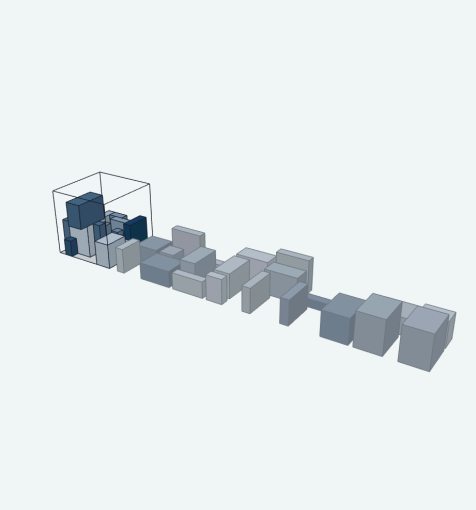
▶ Initial state: Initial state (random)

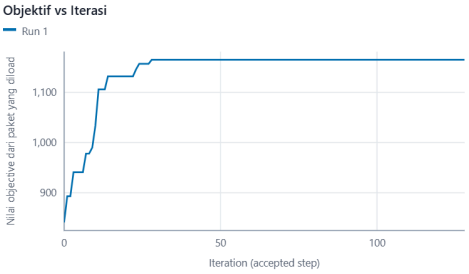
▶ Final state: Final state (terminal)

Final objective value	893
Search duration (s)	0.0543
Banyak iterasi hingga terminasi	1

b. HC with Sideways Move (SMHC)

RUN 1





Hill climbing: sideways

Objective	Nilai total
Parameters	num_cand: 50 / max_side: 100
Base seed	42
Source	inline
Runs	1

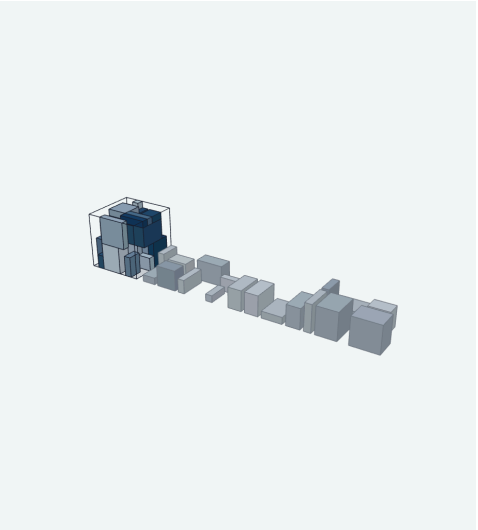
Run	Seed	Final objective	Iterations	Search duration (s)
Run 1	42	1,165	128	6.7154

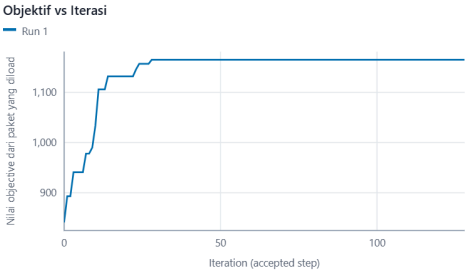
▶ Initial state: Initial state (random)

▶ Final state: Final state (terminal)

Final objective value	1,165
Search duration (s)	6.7154
Banyak iterasi hingga terminasi	128
Maximum pergerakan samping	100

RUN 2





Hill climbing: sideways

Objective	Nilai total
Parameters	num_cand: 50 / max_side: 100
Base seed	42
Source	inline
Runs	1

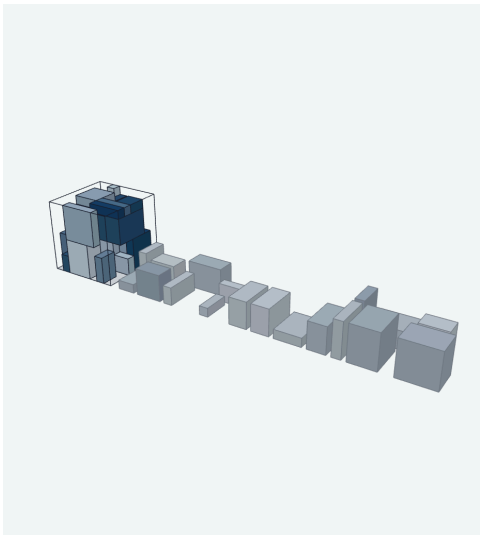
Run	Seed	Final objective	Iterations	Search duration (s)
Run 1	42	1,165	128	7.432

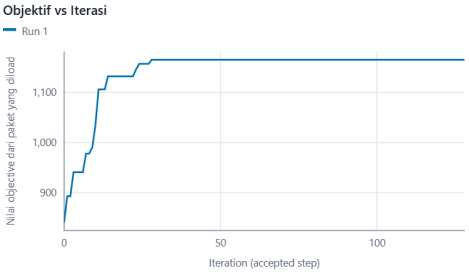
▶ Initial state: Initial state (random)

▶ Final state: Final state (terminal)

Final objective value	1,165
Search duration (s)	7.432
Banyak iterasi hingga terminasi	128
Maximum pergerakan samping	100

RUN 3





Hill climbing: sideways

Objective	Nilai total
Parameters	num_cand: 50 / max_side: 100
Base seed	42
Source	inline
Runs	1

Run	Seed	Final objective	Iterations	Search duration (s)
Run 1	42	1,165	128	5.2196

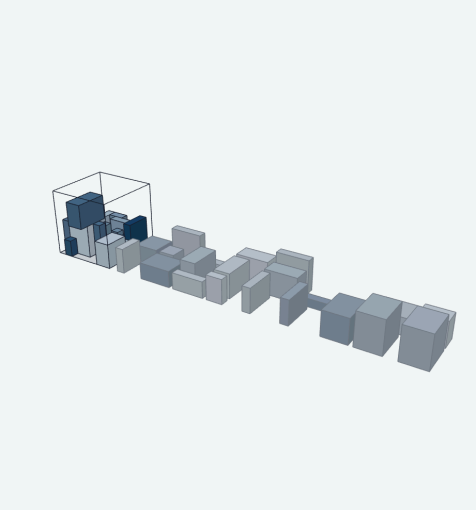
▶ Initial state: Initial state (random)

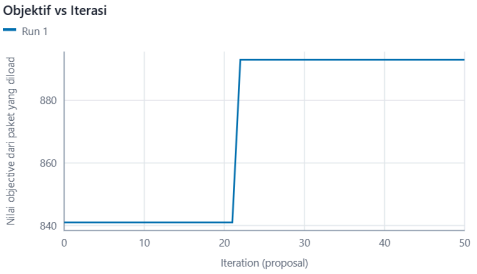
▶ Final state: Final state (terminal)

Final objective value	1,165
Search duration (s)	5.2196
Banyak iterasi hingga terminasi	128
Maximum pergerakan samping	100

c. Stochastic HC (SHC)

RUN 1





Hill climbing: stochastic

Objective	Nilai total
Parameters	nmax: 50
Base seed	42
Source	inline
Runs	1

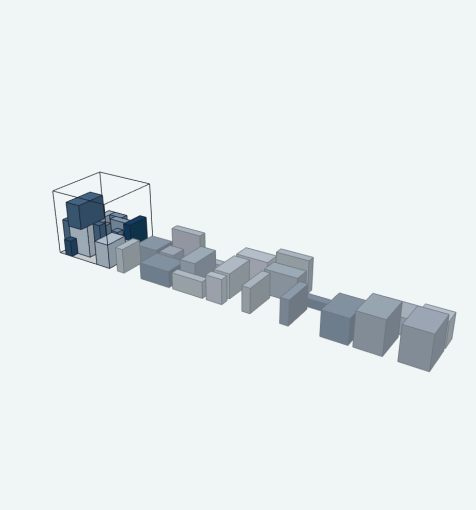
Run	Seed	Final objective	Iterations	Search duration (s)
Run 1	42	893	50	0.0306

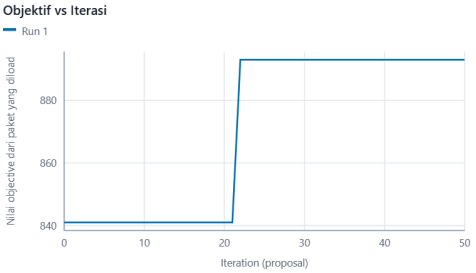
▶ Initial state: Initial state (random)

▶ Final state: Final state (terminal)

Final objective value	893
Search duration (s)	0.0306
Banyak iterasi hingga terminasi	50

RUN 2





Hill climbing: stochastic

Objective	Nilai total
Parameters	nmax: 50
Base seed	42
Source	inline
Runs	1

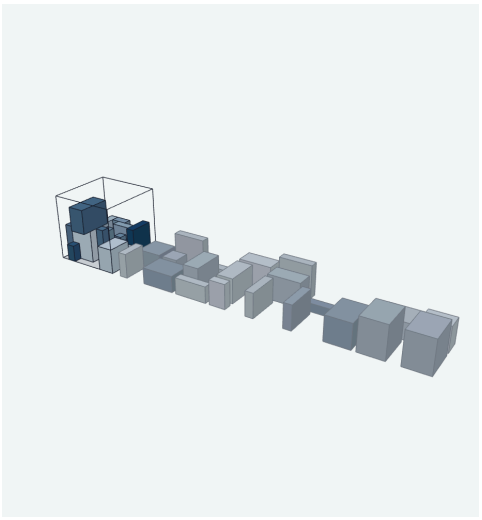
Run	Seed	Final objective	Iterations	Search duration (s)
Run 1	42	893	50	0.0288

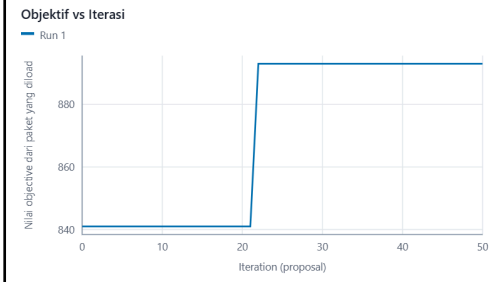
▶ Initial state: Initial state (random)

▶ Final state: Final state (terminal)

Final objective value	893
Search duration (s)	0.0288
Banyak iterasi hingga terminasi	50

RUN 3





Hill climbing: stochastic

Objective	Nilai total
Parameters	nmax: 50
Base seed	42
Source	inline
Runs	1

Run	Seed	Final objective	Iterations	Search duration (s)
Run 1	42	893	50	0.0292

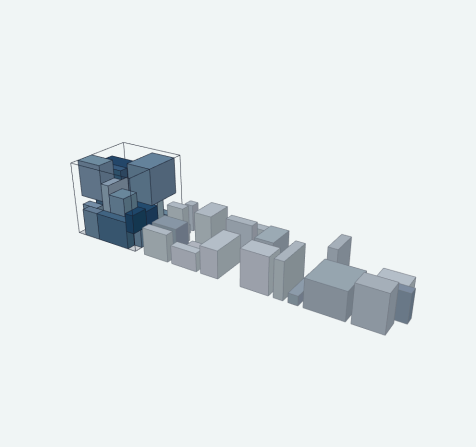
▶ Initial state: Initial state (random)

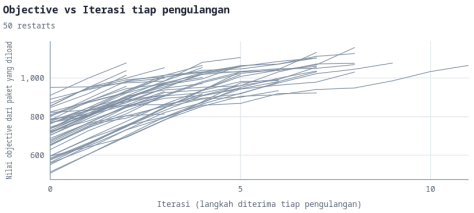
▶ Final state: Final state (terminal)

Final objective value	893
Search duration (s)	0.0292
Banyak iterasi hingga terminasi	50

d. Random Restart HC (RRHC)

RUN 1





Hill climbing: restart

Objective	Nilai total
Parameters	max_restart: 50 / num_cand: 50
Base seed	42
Source	inline
Runs	1

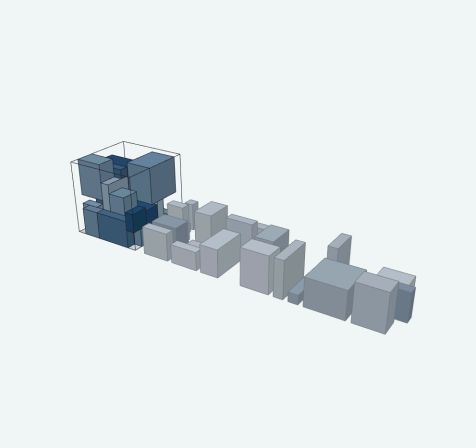
Run	Seed	Final objective	Search duration (s)
Run 1	42	1,157	4.3579

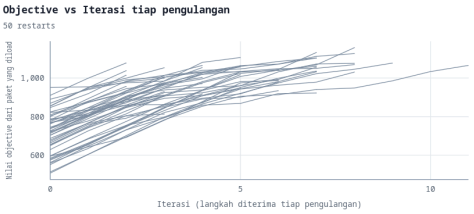
▸ Initial state: Initial state untuk awal percobaan

▸ Final state: Terminal state terbaik dalam percobaan

Final objective value	1,157
Search duration (s)	4.3579
Banyak pengulangan	50
Banyak iterasi tiap pengulangan	5, 5, 3, 8, 3, 7, 3, 7, 3, 2, 7, 3, 4, 7, 8, 5, 6, 3, 2, 2, 5, 9, 6, 3, 6, 8, 7, 4, 5, 2, 5, 3, 3, 4, 7, 5, 2, 4, 7, 8, 2, 4, 6, 8, 4, 3, 2, 7, 5, 11
Pengulangan maksimum	50

RUN 2





Hill climbing: restart

Objective	Nilai total
Parameters	max_restart: 50 / num_cand: 50
Base seed	42
Source	inline
Runs	1

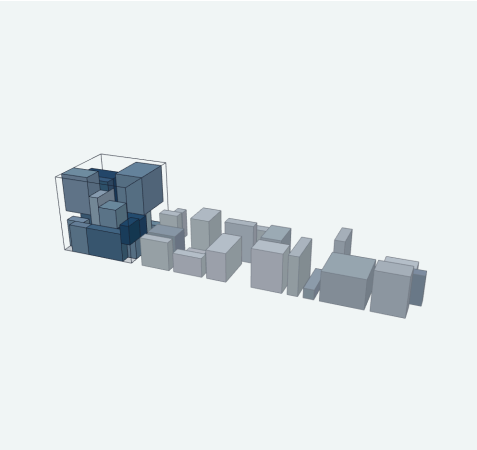
Run	Seed	Final objective	Search duration (s)
Run 1	42	1,157	4.3584

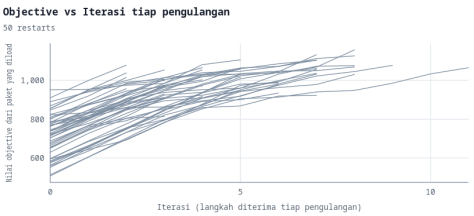
▸ Initial state: Initial state untuk awal percobaan

▸ Final state: Terminal state terbaik dalam percobaan

Final objective value	1,157
Search duration (s)	4.3584
Banyak pengulangan	50
Banyak iterasi tiap pengulangan	5, 5, 3, 8, 3, 7, 3, 7, 3, 2, 7, 3, 4, 7, 8, 5, 6, 3, 2, 2, 5, 9, 6, 3, 6, 8, 7, 4, 5, 2, 5, 3, 3, 4, 7, 5, 2, 4, 7, 8, 2, 4, 6, 8, 4, 3, 2, 7, 5, 11
Pengulangan maksimum	50

RUN 3





Hill climbing: restart

Objective	Nilai total
Parameters	max_restart: 50 / num_cand: 50
Base seed	42
Source	inline
Runs	1

Run	Seed	Final objective	Search duration (s)
Run 1	42	1,157	4.3501

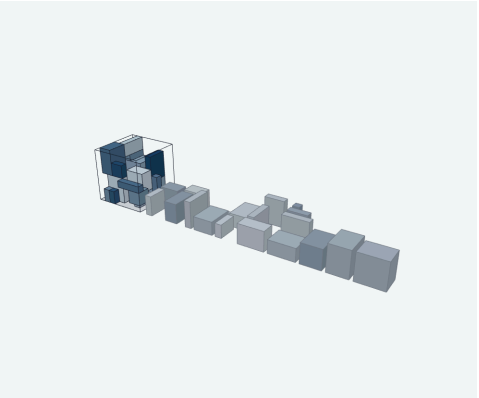
▸ Initial state: Initial state untuk awal percobaan

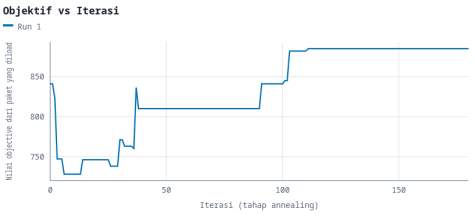
▸ Final state: Terminal state terbaik dalam percobaan

Final objective value	1,157
Search duration (s)	4.3501
Banyak pengulangan	50
Banyak iterasi tiap pengulangan	5, 5, 3, 8, 3, 7, 3, 7, 3, 2, 7, 3, 4, 7, 8, 5, 6, 3, 2, 2, 5, 9, 6, 3, 6, 8, 7, 4, 5, 2, 5, 3, 3, 4, 7, 5, 2, 4, 7, 8, 2, 4, 6, 8, 4, 3, 2, 7, 5, 11
Pengulangan maksimum	50

2. Simulated Annealing

RUN 1





Simulated annealing

Objective	Nilai total
Parameters	t_start: 100 / alpha: 0.95 / t_min: 0.01
Base seed	42
Source	inline
Runs	1

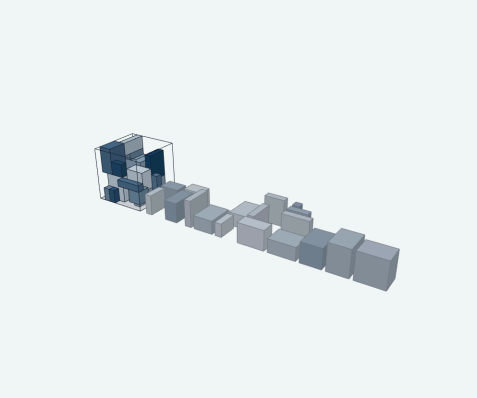
Run	Seed	Final objective	Search duration (s)
Run 1	42	885	0.0448

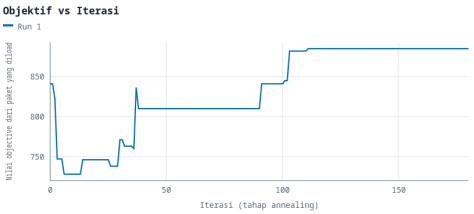
▸ Initial state: Initial state (random)

▸ Final state: Final state

Final objective value	885
Search duration (s)	0.0448
Berapa kali terjebak	85

RUN 2





Simulated annealing

Objective	Nilai total
Parameters	t_start: 100 / alpha: 0.95 / t_min: 0.01
Base seed	42
Source	inline
Runs	1

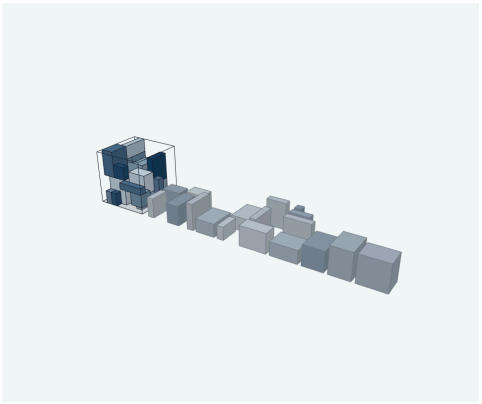
Run	Seed	Final objective	Search duration (s)
Run 1	42	885	0.0452

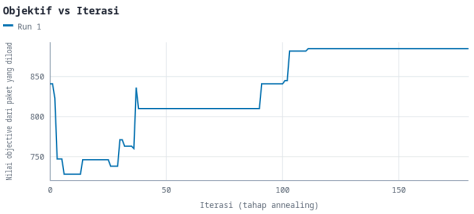
▸ Initial state: Initial state (random)

▸ Final state: Final state

Final objective value	885
Search duration (s)	0.0452
Berapa kali terjebak	85

RUN 3





Simulated annealing

Objective	Nilai total
Parameters	t_start: 100 / alpha: 0.95 / t_min: 0.01
Base seed	42
Source	inline
Runs	1

Run	Seed	Final objective	Search duration (s)
Run 1	42	885	0.0446

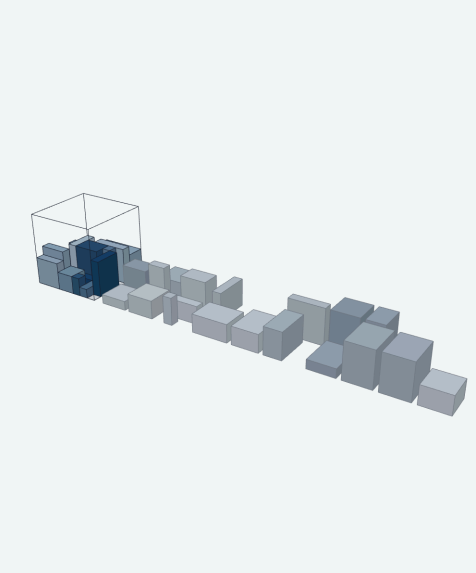
▸ Initial state: Initial state (random)

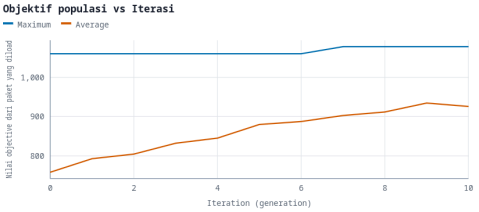
▸ Final state: Final state

Final objective value	885
Search duration (s)	0.0446
Berapa kali terjebak	85

3. Genetic Algorithm (Banyak Iterasi)

RUN 1





Genetic algorithm

Objective	Nilai total
Parameters	pilihan: banyak_iterasi / n: 10
Base seed	42
Source	inline
Runs	9

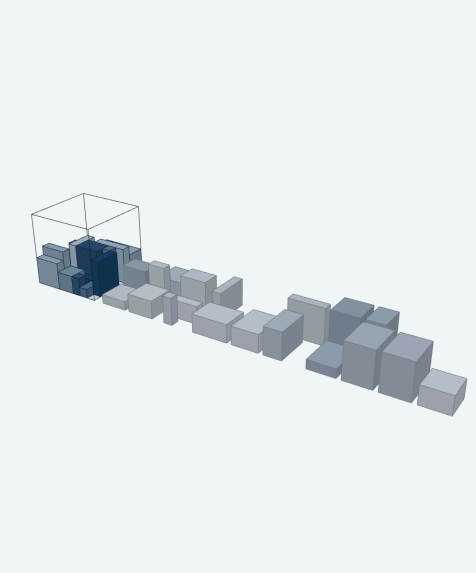
Run	Population	Final objective	Iterations	Search duration (s)
Configuration 2				
Config 2, run 1	41	950	10	0.3056
Config 2, run 2	41	999	10	0.2993
Config 2, run 3	41	916	10	0.2966
Configuration 3				
Config 3, run 1 **Best of nine**	59	1,078	10	0.4634
Config 3, run 2	59	1,060	10	0.4546
Config 3, run 3	59	1,065	10	0.4518

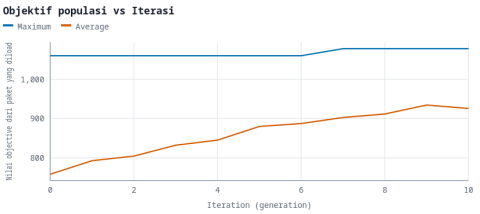
▸ Initial state: Individu terbaik pada populasi awal

▸ Final state: Individu terbaik pada akhir proses

Final objective value	1,078
Search duration (s)	0.4634
Besar populasi	59
Iterasi	10

RUN 2





Genetic algorithm

Objective	Nilai total
Parameters	pilihan: banyak_iterasi / n: 10
Base seed	42
Source	inline
Runs	9

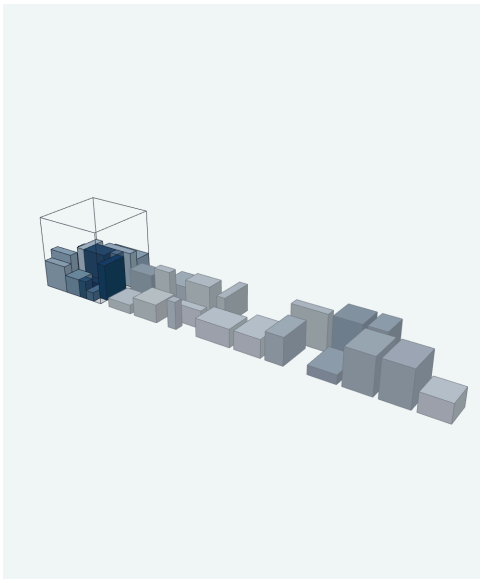
Run	Population	Final objective	Iterations	Search duration (s)
Configuration 2				
Config 2, run 1	41	950	10	0.3033
Config 2, run 2	41	999	10	0.3017
Config 2, run 3	41	916	10	0.2987
Configuration 3				
Config 3, run 1 **Best of nine**	59	1,078	10	0.4534
Config 3, run 2	59	1,060	10	0.4615
Config 3, run 3	59	1,065	10	0.4561

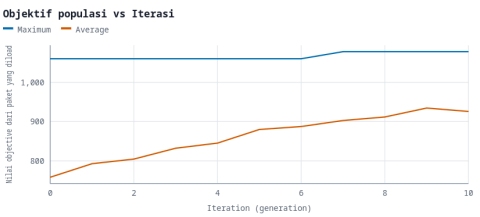
▸ Initial state: Individu terbaik pada populasi awal

▸ Final state: Individu terbaik pada akhir proses

Final objective value	1,078
Search duration (s)	0.4534
Besar populasi	59
Iterasi	10

RUN 3





Genetic algorithm

Objective	Nilai total
Parameters	pilihan: banyak_iterasi / n: 10
Base seed	42
Source	inline
Runs	9

Run	Population	Final objective	Iterations	Search duration (s)
Configuration 2				
Config 2, run 1	41	950	10	0.3049
Config 2, run 2	41	999	10	0.3071
Config 2, run 3	41	916	10	0.2954
Configuration 3				
Config 3, run 1 **Best of nine**	59	1,078	10	0.4556
Config 3, run 2	59	1,060	10	0.4593
Config 3, run 3	59	1,065	10	0.45

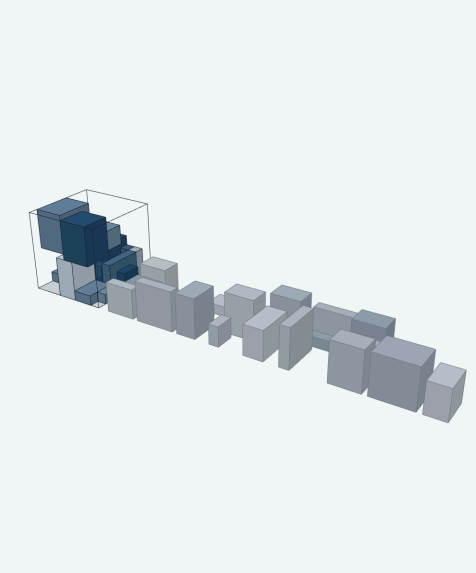
▸ Initial state: Individu terbaik pada populasi awal

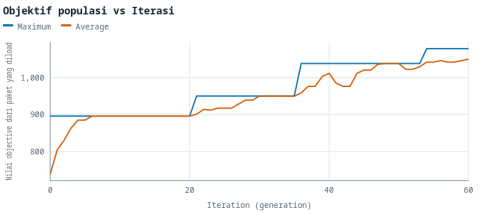
▸ Final state: Individu terbaik pada akhir proses

Final objective value	1,078
Search duration (s)	0.4556
Besar populasi	59
Iterasi	10

4. Genetic Algorithm (Jumlah Populasi)

RUN 1





Genetic algorithm

Objective	Nilai total
Parameters	pilihan: jumlah_populasi / n: 10
Base seed	42
Source	inline
Runs	9

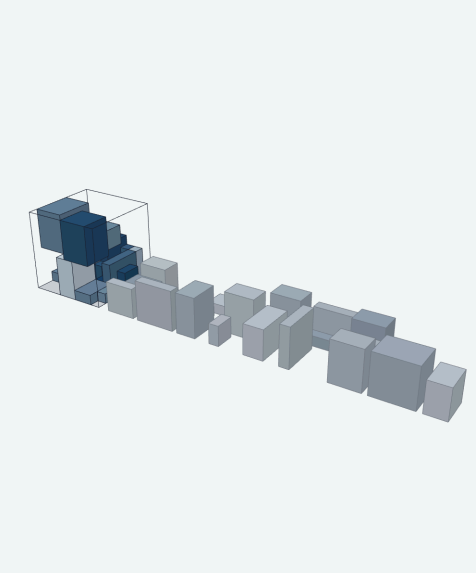
Run	Population	Final objective	Iterations	Search duration (s)
Configuration 1				
Config 1, run 1	10	912	1	0.0093
Config 1, run 2	10	896	1	0.0119
Config 1, run 3	10	896	1	0.0087
Configuration 2				
Config 2, run 1	10	1,025	60	0.4428
Config 2, run 2	10	1,032	60	0.4463
Config 2, run 3 **Best of nine**	10	1,078	60	0.4549

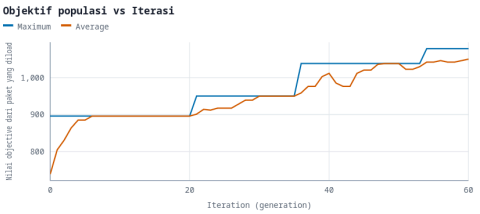
▸ Initial state: Individu terbaik pada populasi awal

▸ Final state: Individu terbaik pada akhir proses

Final objective value	1,078
Search duration (s)	0.4549
Besar populasi	10
Iterasi	60

RUN 2





Genetic algorithm

Objective	Nilai total
Parameters	pilihan: jumlah_populasi / n: 10
Base seed	42
Source	inline
Runs	9

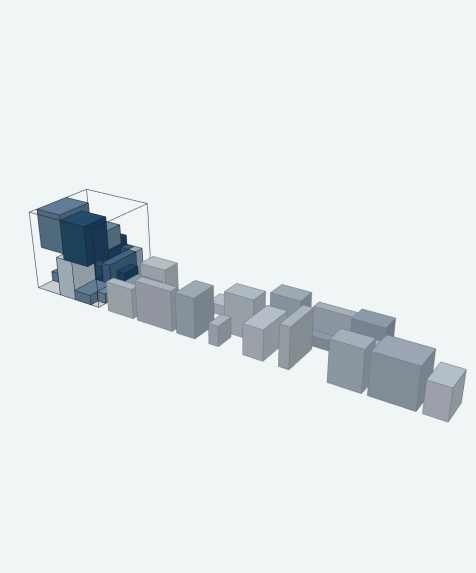
Run	Population	Final objective	Iterations	Search duration (s)
Configuration 1				
Config 1, run 1	10	912	1	0.009
Config 1, run 2	10	896	1	0.0087
Config 1, run 3	10	896	1	0.0085
Configuration 2				
Config 2, run 1	10	1,025	60	0.4409
Config 2, run 2	10	1,032	60	0.4487
Config 2, run 3 Best of nine	10	1,078	60	0.4529

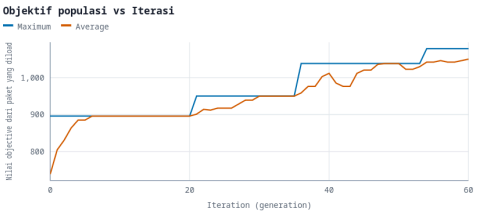
▸ Initial state: Individu terbaik pada populasi awal

▸ Final state: Individu terbaik pada akhir proses

Final objective value	1,078
Search duration (s)	0.4529
Besar populasi	10
Iterasi	60

RUN 3





Genetic algorithm

Objective	Nilai total
Parameters	pilihan: jumlah_populasi / n: 10
Base seed	42
Source	inline
Runs	9

Run	Population	Final objective	Iterations	Search duration (s)
Configuration 1				
Config 1, run 1	10	912	1	0.009
Config 1, run 2	10	896	1	0.0088
Config 1, run 3	10	896	1	0.0086
Configuration 2				
Config 2, run 1	10	1,025	60	0.4729
Config 2, run 2	10	1,032	60	0.4442
Config 2, run 3 Best of nine	10	1,078	60	0.4602

▸ Initial state: Individu terbaik pada populasi awal

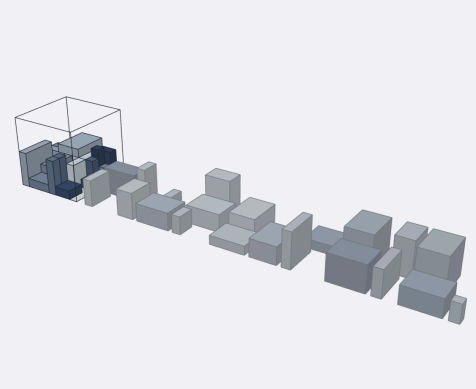
▸ Final state: Individu terbaik pada akhir proses

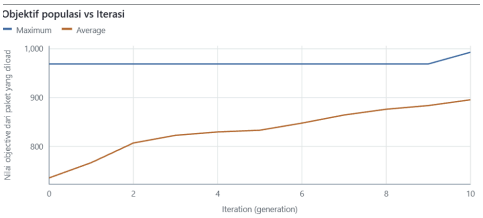
Final objective value	1,078
Search duration (s)	0.4602
Besar populasi	10
Iterasi	60

5. Objective Value 1

Variabel Terkontrol: Jenis Algoritma (Genetic Algorithm), Banyak Iterasi (10), dan Testcase yang sama

RUN





Genetic algorithm

Objective	Nilai total
Parameters	pilihan: banyak_iterasi / n: 10
Base seed	42
Source	inline
Runs	9

Run	Population	Final objective	Iterations	Search duration (s)
Configuration 1				
Config 1, run 1	82	969	10	1.2762
Config 1, run 2	82	982	10	1.4149
Config 1, run 3 **Best of nine**	82	993	10	1.5402
Configuration 2				
Config 2, run 1	11	894	10	0.1792
Config 2, run 2	11	887	10	0.1746
Config 2, run 3	11	921	10	0.1869

▶ Initial state: Individu terbaik pada populasi awal

▶ Final state: Individu terbaik pada akhir proses

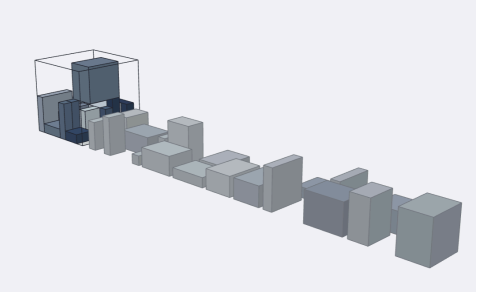
Final objective value	993
Search duration (s)	1.5402
Besar populasi	82
Iterasi	10

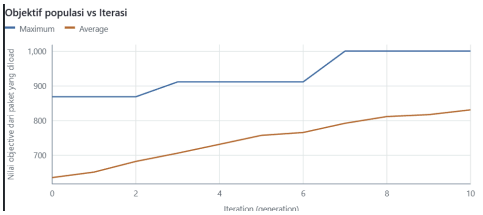
Configuration 3				
Config 3, run 1	14	944	10	0.2406
Config 3, run 2	14	937	10	0.2635
Config 3, run 3	14	937	10	0.2494

6. Objective Value 2

Variabel Terkontrol: Jenis Algoritma (Genetic Algorithm), Banyak Iterasi (10), dan Testcase yang sama

RUN





Genetic algorithm

Objective	Nilai Total - Penalti ETA
Parameters	pilihan: banyak_iterasi / n: 10
Base seed	42
Source	inline
Runs	9

Run	Population	Final objective	Iterations	Search duration (s)
Configuration 1				
Config 1, run 1	82	909	10	1.6162
Config 1, run 2	82	891	10	1.7517
Config 1, run 3 **Best of nine**	82	1,001	10	1.6924
Configuration 2				
Config 2, run 1	17	858	10	0.3475
Config 2, run 2	17	807	10	0.3095
Config 2, run 3	17	807	10	0.3331

▶ Initial state: Individu terbaik pada populasi awal

▶ Final state: Individu terbaik pada akhir proses

Final objective value	1,001
Search duration (s)	1.6924
Besar populasi	82
Iterasi	10

Configuration 3				
Config 3, run 1	67	864	10	1.3419
Config 3, run 2	67	864	10	1.3748
Config 3, run 3	67	864	10	1.2635

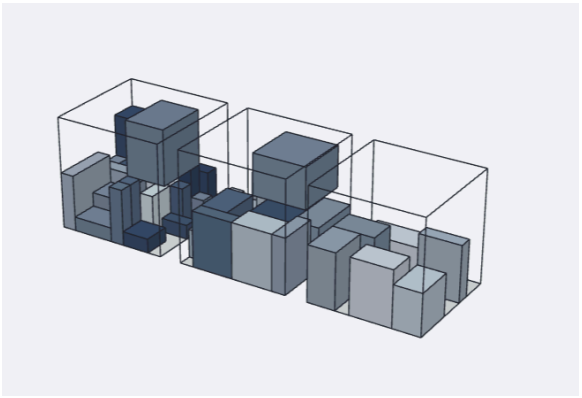
7. Multitruk

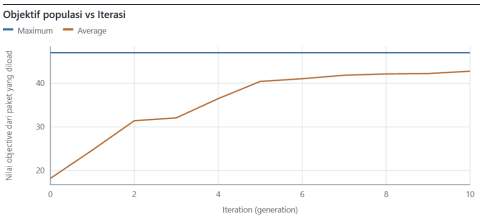
Testcase:

```
GA / SA
8 8 8 104
8 8 8 104
8 8 8 104
banyak_iterasi
10
40
2 3 1 6 43 0 2
1 3 1 6 48 1 5
3 1 3 5 12 0 2
3 2 3 10 79 0 2
4 2 3 4 89 0 1
2 1 1 7 51 0 1
2 2 2 3 48 0 3
3 2 3 4 8 1 5
1 4 4 4 26 0 0
2 4 1 3 11 1 4
4 2 3 10 45 0 1
1 1 1 4 11 0 0
1 3 1 8 20 0 2
1 2 3 4 2 0 4
2 1 2 4 18 0 5
2 2 1 5 91 1 4
2 4 3 8 12 0 0
4 3 2 1 3 0 2
3 4 1 4 19 0 0
3 4 1 4 24 0 2
2 3 4 5 3 0 1
3 2 4 7 45 0 1
1 4 4 7 24 0 4
1 4 3 10 70 1 4
1 1 4 4 97 0 3
4 1 1 10 68 1 1
4 3 3 4 88 0 1
3 4 4 3 54 0 1
4 3 1 5 34 0 3
2 4 3 7 33 0 0
3 1 3 1 32 0 4
1 1 2 9 84 0 2
2 3 1 4 52 0 4
4 1 3 10 96 0 3
1 1 3 5 76 0 5
2 3 4 5 72 0 2
4 4 3 4 50 0 2
```

```
1 1 4 5 62 1 2
1 2 1 7 25 0 2
3 3 2 3 3 0 2
50
```

RUN - Genetic Algorithm





Genetic algorithm

Objective	Nilai Total - Penalti ETA
Parameters	pilihan: banyak_iterasi / n: 10
Base seed	42
Source	inline
Runs	9

Run	Population	Final objective	Iterations	Search duration (s)
Configuration 1				
Config 1, run 1 **Best of nine**	100	1,428	10	2.3451
Config 1, run 2	100	1,428	10	2.5621
Config 1, run 3	100	1,428	10	2.4588
Configuration 2				
Config 2, run 1	43	1,428	10	0.8851
Config 2, run 2	43	1,428	10	0.8786
Config 2, run 3	43	1,428	10	0.8351

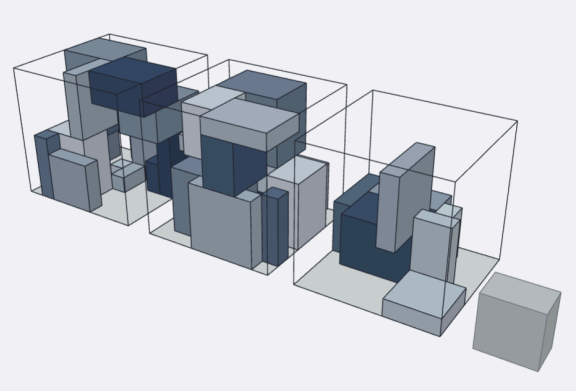
▶ Initial state: Individu terbaik pada populasi awal

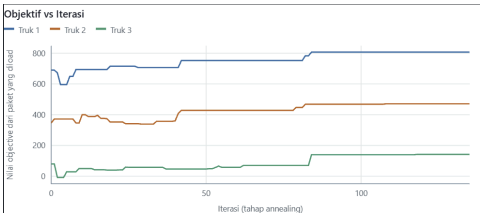
▶ Final state: Individu terbaik pada akhir proses

Final objective value	1,428
Search duration (s)	2.3451
Besar populasi	100
Iterasi	10

Configuration 3				
Config 3, run 1	37	1,428	10	1.7525
Config 3, run 2	37	1,428	10	1.7474
Config 3, run 3	37	1,428	10	1.6073

RUN - Simulated Annealing





Simulated annealing

Objective	Nilai Total - Penalti ETA
Parameters	t_start: 100 / alpha: 0.95 / t_min: 0.1
Base seed	42
Source	inline
Runs	1

Run	Seed	Final objective	Search duration (s)
Run 1	42	1,420	0.1165

▶ Initial state: Initial state (random)

▶ Final state: Final state

Final objective value	1,420
Search duration (s)	0.1165
Berapa kali terjebak	202

8. Ringkasan Perbandingan

Tabel 1 Perbandingan Semua Algoritma

Perbandingan	Nilai	Jumlah	Durasi	Peringkat

	Objective Akhir	Iterasi		
Steepest Ascent HC	893	1	0.0546	4
HC with Sideways Move	1165	128	6.4557	1
Stochastic HC	893	50	0.0295	4
Random Restart HC	1157	248	4.3555	2
Simulated Annealing	885	180	0.0449	6
Genetic Algorithm	1078	10	0.4575	3

Tabel 2 Perbandingan dua Objective Value

Perbandingan	Nilai Objective Akhir (Best of 9)	Durasi
Objective Value 1	993	1.4104
Objective Value 2	1001	1.6868

Tabel 3 Perbandingan dua algoritma untuk Multitruk

Perbandingan	Nilai Objective Akhir	Durasi
Multitruk 1 (GA)	1428	0.8351 - 2.5621
Multitruk 2 (SA)	1420	0.1165

9. Pembahasan Perbandingan

a. Tabel 1 - Perbandingan enam Algoritma

Berdasarkan hasil pengujian, HC with Sideways Move menghasilkan nilai objective tertinggi (1,165), diikuti Random Restart HC (1,157) dan Genetic Algorithm (1,078). Steepest Ascent HC dan Stochastic HC hanya mencapai 893, sedangkan Simulated Annealing menghasilkan nilai terendah (885). Perbedaan ini terutama dipengaruhi oleh cara tiap algoritma menghadapi **plateau**.

Pada persoalan ini, banyak operasi neighbor seperti menggeser atau memutar paket tidak mengubah paket yang berada di dalam truk, sehingga menghasilkan state dengan nilai yang sama. Steepest Ascent HC dan Stochastic HC hanya menerima neighbor yang lebih baik, sehingga keduanya berhenti begitu tidak ada paket yang dapat langsung ditambahkan dan terjebak pada maksimum lokal. Sideways Move mengatasinya dengan mengizinkan perpindahan mendatar, sedangkan Random Restart HC mengatasinya dengan memulai pencarian dari state awal yang berbeda. Keunggulan kualitas ini dibayar dengan waktu eksekusi yang jauh lebih lama, karena pencarian berlangsung lebih panjang (pada

Sideways Move) atau diulang berkali-kali (pada Random Restart HC) dengan puluhan neighbor yang dievaluasi pada setiap iterasi.

Simulated Annealing menghasilkan nilai terendah karena temperatur awal yang digunakan terlalu besar dibandingkan selisih nilai antar-state. Akibatnya, langkah yang lebih buruk hampir selalu diterima di awal pencarian dan nilai sempat turun cukup jauh. Pendinginan dengan alpha 0.95 juga berlangsung terlalu cepat sehingga algoritma tidak sempat pulih. Sementara itu, Genetic Algorithm berada di posisi tengah, dengan kualitas yang bergantung pada jumlah populasi dan banyak iterasi yang digunakan.

b. Tabel 2 - Perbandingan dua Objective Value

Nilai Objective Value 1 dan Objective Value 2 tidak dapat dibandingkan secara langsung, karena Objective Value 2 selalu dikurangi penalti ETA yang bernilai minimum 100 meskipun tidak ada pelanggaran. Oleh karena itu, nilai terbaik Objective Value 2 yang sedikit lebih tinggi (1,001 dibandingkan 993) **tidak berarti hasilnya lebih baik**.

Pada kedua objective, populasi yang lebih besar cenderung menghasilkan nilai yang lebih baik karena keragaman individu yang tersedia untuk crossover dan mutation lebih luas. Hasil Objective Value 2 juga lebih bervariasi pada populasi yang sama. Kemungkinan penyebabnya adalah penalti yang memperhitungkan jumlah pasangan paket yang melanggar urutan ETA, sehingga perubahan kecil pada susunan paket dapat menurunkan nilai dengan tajam. Selain itu, Objective Value 2 membutuhkan waktu eksekusi yang lebih lama yang disebabkan oleh setiap evaluasi harus memeriksa urutan ETA antar-paket. Sebagai catatan, jumlah populasi pada beberapa konfigurasi ditentukan secara acak, sehingga hanya konfigurasi dengan populasi yang sama yang dapat dibandingkan secara langsung.

c. Tabel 3 - Perbandingan dua Algoritma pada multitruk

Pada pengujian multitruk, Genetic Algorithm menghasilkan nilai yang sama (1,428) pada seluruh percobaan, terlepas dari jumlah populasi yang digunakan. Nilai tersebut merupakan nilai maksimum yang mungkin, karena total nilai seluruh paket dikurangi penalti minimum pada tiga truk menghasilkan angka yang sama. Artinya, seluruh paket berhasil dimuat tanpa pelanggaran urutan ETA.

Pada kasus ini, jumlah populasi hanya memengaruhi waktu eksekusi dan bukan kualitas solusi, dan waktu yang dibutuhkan tidak selalu meningkat seiring bertambahnya populasi. Simulated Annealing menghasilkan nilai yang sedikit lebih rendah (1,420) karena satu paket tidak berhasil dimuat. Selisihnya sama dengan nilai paket tersebut, sesuai dengan visualisasi 3D yang menunjukkan satu paket tertinggal di luar truk ketiga. Namun, waktu eksekusinya jauh lebih singkat, sehingga selisih kualitas yang kecil ini sebanding dengan penghematan waktu yang diperoleh. Pemrosesan truk secara sekuensial juga cukup efektif pada kasus ini karena kapasitas total truk masih mencukupi untuk seluruh paket. Truk pertama terisi paling padat, sedangkan truk terakhir hanya menerima sisa paket.

Secara keseluruhan, perbandingan ini menunjukkan adanya *trade-off* antara kualitas solusi dan waktu eksekusi. HC with Sideways Move dan Random Restart HC menghasilkan solusi terbaik pada kasus satu truk dengan waktu paling lama, sedangkan Steepest Ascent HC dan Stochastic HC sangat cepat tetapi terjebak

pada maksimum lokal. Simulated Annealing sangat bergantung pada pengaturan temperatur awal dan faktor pendinginan, sehingga hasilnya kurang baik pada parameter yang digunakan tetapi cukup efektif pada kasus multitruk. Genetic Algorithm menghasilkan solusi yang membaik seiring bertambahnya populasi dan banyak iterasi. Fungsi objektif kedua membuat pencarian lebih sulit karena adanya penalti ETA, sedangkan pendekatan multitruk berjalan dengan baik selama kapasitas total truk mencukupi.

Sebagai catatan, seluruh pengujian menggunakan satu testcase dan seed yang sama (42), sehingga hasil ini belum cukup untuk digeneralisasi pada kasus lain.

BAB IV
KESIMPULAN DAN SARAN

1. Kesimpulan

Berdasarkan hasil pengujian, seluruh algoritma yang diimplementasikan berhasil menghasilkan konfigurasi muatan truk yang valid, tetapi dengan kualitas dan waktu eksekusi yang berbeda-beda. Pada kasus satu truk, HC with Sideways Move dan Random Restart HC menghasilkan solusi terbaik dengan waktu paling lama, sedangkan Steepest Ascent HC dan Stochastic HC sangat cepat tetapi terjebak pada maksimum lokal akibat plateau. Simulated Annealing sangat dipengaruhi pengaturan temperatur awal dan faktor pendinginan, sedangkan Genetic Algorithm menghasilkan solusi yang membaik seiring bertambahnya populasi dan banyak iterasi. Objective Function 2 membuat pencarian lebih sulit dan lebih lambat karena adanya penalti ETA, sementara pendekatan multitruk berjalan dengan baik selama kapasitas total truk mencukupi.

Secara keseluruhan, tidak ada satu algoritma yang unggul dalam semua aspek karena terdapat *trade-off* antara kualitas solusi dan waktu eksekusi.

2. Saran

- Pengujian sebaiknya dilakukan dengan lebih banyak testcase dan seed yang berbeda, lalu hasilnya dirata-ratakan agar perbandingan antar algoritma lebih dapat diandalkan.
- Fungsi objektif dapat ditambahkan komponen sekunder, misalnya sisa kapasitas atau ruang kosong pada truk, agar state dengan nilai paket yang sama tetap dapat dibedakan dan plateau berkurang.
- Operasi neighbor dapat dibuat lebih terarah, misalnya dengan memprioritaskan pemasukan paket yang memiliki nilai tinggi dibandingkan beratnya.
- Pada kasus multitruk, pembagian paket antar-truk dapat dikembangkan agar tidak hanya bergantung pada urutan truk, sehingga hasilnya tetap baik ketika kapasitas total truk lebih terbatas.

PEMBAGIAN TUGAS

NAMA - NIM	TUGAS
Renuno Yuqa Frinardi - 13524080	Membuat algoritma Hill Climbing, bonus *objective function*, *parser* Hill Climbing, dan *backend*
Valentino Daniel Kusumo - 13524104	Membuat tipe data, *utils function*, *parser*, algoritma *simulated annealing*, dan *frontend*
Leonardus Brain Fatolosja - 13524146	Membuat algoritma *Genetic Algorithm*, GUI (3D View dan Chart), dan *parser*

REFERENSI

[1] S. J. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ: Pearson, 2020.

[2] Tim Pengajar IF3170 Inteligensi Artifisial, "Beyond Classical Search," Slide Kuliah IF3170 Inteligensi Artifisial Semester I TA 2026/2027, Program Studi Teknik Informatika, Institut Teknologi Bandung, 2026.

[3] GeeksforGeeks, "What is Simulated Annealing?," GeeksforGeeks, Sep. 1, 2026. [Daring]. Tersedia: https://www.geeksforgeeks.org/artificial-intelligence/what-is-simulated-annealing/. [Diakses: 9 Okt 2026].

[4] GeeksforGeeks, "Genetic Algorithms," GeeksforGeeks, Dec. 2023. [Daring]. Tersedia: https://www.geeksforgeeks.org/dsa/genetic-algorithms/. [Diakses: 16 Okt 2026]

[5] Algorithm Afternoon, "Elitist Genetic Algorithm," Algorithm Afternoon. [Daring]. Tersedia: https://algorithmafternoon.com/genetic/elitist_genetic_algorithm/ [Diakses: 16 Okt 2026].

[6] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2 Sep. 2020. [Daring]. Tersedia: https://youtu.be/oGTa9kGE3P8. [Diakses: 17 Okt. 2026].

[7] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2 Sep. 2020. [Daring]. Tersedia: https://youtu.be/_6rla37QorM. [Diakses: 17 Okt. 2026].

[8] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2 Sep. 2020. [Daring]. Tersedia: https://youtu.be/L0b0CN3Fds8. [Diakses: 17 Okt. 2026].

[9] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2 Sep. 2020. [Daring]. Tersedia: https://youtu.be/AdhG7HnhYrk. [Diakses: 9 Okt. 2026].

[10] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 4 Sep. 2020. [Daring]. Tersedia: https://youtu.be/aI41TGo9J9c. [Diakses: 16 Okt. 2026].

[11] Mozilla, “WebGL: 2D and 3D Graphics for the Web,” MDN Web Docs, Mar. 04, 2019. https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API [accessed Oct. 20, 2026].

FORMAT DEKLARASI PENGGUNAAN AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Renuno Yuqa Frinardi (13524080), Valentino Daniel Kusumo (13524104), Leonardus Brain Fatolosja (13524146)
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Intelegensi Artifisial
Judul Tugas/Proposal : IF3170 INTELIGENSI ARTIFISIAL LAPORAN TUGAS BESAR 1

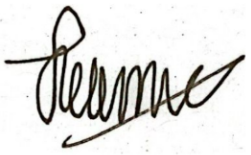
Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, DeepL, Quillbot, Grammarlybot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	**Tidak**	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	**Tidak**	-
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	**Ya**	AI digunakan dalam membantu diskusi penyusunan arsitektur sistem GUI, mulai dari *backend*, *frontend*, dan *message passing* antara kedua *endpoint*
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	**Tidak**	-

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Renuno Yuqa Frinardi

NIM 13524080



Valentino Daniel Kusumo

NIM 13524104



Leonardus Brain Fatolosja

NIM 13524146

LAMPIRAN

Github Repository: https://github.com/renuno-frinardi/IF3170_G28_MindfulnessAI